GitHub スター上位レポート（過去 365 日）

2026-10-30 作成 ／ 対象: 2025-10-30 以降に作成されたリポジトリ
テーマ: WebLLM

#	区分	リポジトリ	★	一言で
1	テーマ	hooosberg/WitNote	568	Ollama やクラウド API と連携できる、ローカルファースト志向の Electron 製 Markdown 二画面エディタ
2	テーマ	RunanywhereAI/on-device-browser-agent	305	Nanobrowser をフォークし、AI モデルを拡張機能内で動かすローカル完結型の Chrome 向け Web エージェント
3	テーマ	YuriCrystal/ai-avatar-bot	204	一行の script タグで任意のサイトに埋め込める、Live2D／VRM 対応の音声 AI アバターウィジェットと運営管理画面
4	テーマ	activated-intelligence/voice-chat	197	音声認識・LLM・音声合成をすべてブラウザ内で動かす、ハンズフリーのAI音声チャットアプリ
5	テーマ	hamzaydia/verifyfetch	160	ブラウザで巨大なAIモデル等を、定数メモリでハッシュ検証しつつ途中から再開ダウンロードできるTypeScriptライブラリ
6	テーマ	noumena-labs/Sipp	122	GGUFモデルをブラウザ上のWebGPUで動かせ、ローカル推論とクラウドゲートウェイ経由の推論を同じAPIで扱えるRust製AI推論SDK
7	テーマ	darshi1337/apogee	99	記事・動画・PDF・DOCXなどをブラウザ内のローカルAIで要約する、プライバシー重視のChrome/Firefox拡張機能
8	テーマ	FMXExpress/PasClaw	64	Delphi/Free PascalのObject Pascalで書かれた、単一バイナリで動く軽量な個人向けAIエージェントフレームワーク
9	テーマ	LocalMode-AI/LocalMode	64	ブラウザ内だけで埋め込み・ベクトル検索・LLMチャット・音声・画像処理などを動かすローカルファーストAIツールキット
10	テーマ	luskB/MCP-WebLLM	57	ChatGPTやDeepSeekなど網頁版AIチャットからローカルのMCPツールを呼び出せるようにするChrome/Edge拡張機能
11	テーマ	webllm/renderify	33	LLMが生成したJSX/TSXをビルド不要でブラウザ上で直接実行・描画するランタイム優先のUIエンジン
12	テーマ	Kriswd/webllm-gateway	32	Qwen や DeepSeek の Web 版チャットを、OpenAI／Anthropic 互換でツール呼び出し可能な API に変換するゲートウェイです。
13	テーマ	salpbes/GomeraX	28	ブラウザ内で動くローカルLLMのBIMアシスタントを備えた、Open BIM Componentsベースの実験的IFCビューア
14	テーマ	webllm/webblackbox	15	Webアプリの操作・通信・ログ等を常時記録し、後から再生・解析できるフライトレコーダー兼タイムトラベルデバッガー

15	テーマ	116-Labs/OfflineCV	11	履歴書PDFが汎用テキスト抽出器にどう読まれるかを、ブラウザ内だけで診断できるログイン不要の求職ワークベンチ
16	テーマ	TechMitten/Origami-AI	11	PDFスライドをAIナレーション付き動画に変換するなど、WebGPUでAI処理をすべてブラウザ内で完結させるAIスタジオ
17	テーマ	webllm-org/ai-blocks	10	WebLLM上で動く、サーバーもAPIキーも不要なReact向けAIコンポーネント集（shadcn形式）
18	テーマ	kyrillosishak/Domicile	10	ベクトルDB・埋め込み・RAG・ローカルLLMをすべてブラウザ内で動かし、データを端末外に出さないプライベートAIスタック
19	テーマ	soulbah/regeste	9	文書の解析から検索・回答生成までをブラウザ内で完結させる、プライバシー重視の文書チャット（RAG）アプリ
20	テーマ	KREASIOKA/KREASYS	7	ブラウザだけで動くサーバーレスなAIエージェント用IDE。WebGPUによるローカルLLM推論とTelegram連携に対応する

テーマ 1. hooosberg/WitNote

https://github.com/hooosberg/WitNote

★ 568 ／ フォーク 36 ／ 言語 TypeScript ／ 作成 2025-12-17
該当テーマ: WebLLM

Ollama やクラウド API と連携できる、ローカルファースト志向の Electron 製 Markdown 二画面エディタ

概要

WitNote は、ローカルの .md / .txt ファイルやフォルダをそのまま開き、左で書いて右でライブプレビューする二画面 Markdown エディタです。ファイルはすべてローカルディスクに残り、AI は百科事典・翻訳・執筆補助といった軽量な相棒として、Ollama のローカルモデルや DeepSeek・OpenAI などのクラウド API に接続して使います。複雑化する AI 執筆ツールに対し、シンプルさと「すべてはファイル」という設計を打ち出している点が特徴です。オープンソース版は Windows / Linux 向けに継続開発され、macOS 向けは Swift 製のネイティブ版が Mac App Store で配布されています。

要点

- 左編集・右プレビューの二画面 Markdown エディタ
- データベースを使わずフォルダをそのまま作業場所にする
- Ollama・クラウド API・WebLLM の AI エンジンに対応
- OSS版は Electron 製で Windows/Linux の x64・ARM64 を配布
- Mac 版は Swift ネイティブへ移行し App Store で配布

何をするものか

WitNote は「Smart Core, Simple Form」を掲げる、軽量でローカルファーストな Markdown 執筆ツールです。README では、AI ツールが複雑・肥大化する中で「速く、気が散らず、洗練された Markdown の二画面編集」に立ち返ることを理念としています。任意のローカルフォルダをワークスペースとして開き、独自形式やデータベースによるロックインを避けて、すべてをファイルとして扱います。AI は侵襲的なエージェントではなく、オフラインの百科事典、翻訳者、執筆アシスタントとして位置付けられています。プライバシー面では、ノートや秘密鍵を収集・アップロードしないと明記されています。

主な機能

GFM と LaTeX 数式に対応した二画面ライブプレビューがあり、複数行の数式、揃え、行列、コードハイライトを扱えます。セキュリティ面では DOMPurify によるサニタイズを組み込み、リンクは既定のブラウザで開く仕組みです。AI 連携では Ollama とクラウド API に接続でき、ワンクリックでコンテキストをクリアできます。UI はすりガラス調のカード型で、Light / Dark / Tea のテーマとフォーカスモードを備えています。package.json のファイル関連付けやコンポーネント構成から、PDF・docx・画像のビューア機能も含まれていることがわかります。

仕組み・アーキテクチャ

Electron アプリとして、electron/main.ts と preload.ts のメインプロセス側と、React による src 以下のレンダラ側に分かれています。AI 関連は src/engines に OpenAIEngine.ts と WebLLMEngine.ts、src/services に OllamaService.ts と inlineAiEdit.ts が置かれ、engineStore.ts で切り替えを管理しているとみられます。スクリーンショット名にも「三種AI引擎」とあり、Ollama・クラウド API・WebLLM の3系統を想定した構成です。ただし Windows / Linux のビルドスクリプトは DISABLE_WEBLLM=true を指定しており、WebLLM は無効化されてビルドされます。その他、Editor、ChatPanel、DiffPreview、AutocompletePopup などのコンポーネントがあり、インライン AI 編集や差分プレビュー、補完機能を持つことがうかがえます。

技術構成

言語は TypeScript で、UI は React 18、ビルドは Vite 5 と vite-plugin-electron、パッケージングは electron-builder を使用しています。Markdown のレンダリングには marked、数式には katex、HTML サニタイズには dompurify を使っています。ブラウザ内 LLM 推論には @mlc-ai/web-llm、ファイル監視には chokidar、設定保存には electron-store を採用しています。PDF 表示には pdfjs-dist と react-pdf、Word 文書には mammoth を使用しています。多言語化は i18next と react-i18next で行い、locales には日本語を含む英・中（簡体・繁体）・韓・独・仏・西の各言語ファイルがあります。テストには vitest と jsdom が使われています。

使い方・導入方法

Windows と Linux のユーザーは GitHub Releases からバイナリを入手できます。v2.0.1 以降は x86_64 と ARM64 の両方に対応し、Windows はセットアップインストーラ（.exe）、Linux は AppImage と Debian / Ubuntu 向けの .deb が用意されています。macOS ユーザーは Mac App Store から Swift 製のネイティブ版を入手できます。ソースからビルドする場合は、リポジトリを clone して npm install を実行し、npm run dev で開発モードを起動します。パッケージ作成には npm run build:win や npm run build:linux を使います。README によれば、Mac で Electron 版を使いたい場合もソースからビルドして実行できます。

注目ポイント・注意点

Apple プラットフォーム向けは Swift / SwiftUI のネイティブ実装へ移行済みですが、このリポジトリの中身は Electron 版です。Swift 版のソースが公開されているかは README からは不明です。また、GitHub メタデータ上のライセンスは「不明」ですが、README と package.json には MIT と記載されています。README 冒頭にはスポンサーである API 中継サービスの紹介が大きく掲載されています。クラウド API を使う場合、送信内容は外部サービスに渡るため、完全オフラインで使いたいなら Ollama の利用が前提になります。なお、トピックには WebLLM が含まれていますが、Windows / Linux ビルドでは無効化されている点に注意が必要です。

技術構成: TypeScript、Electron、React、Vite、electron-builder、marked、KaTeX、DOMPurify、@mlc-ai/web-llm、Ollama、chokidar、electron-store、pdfjs-dist、react-pdf、mammoth、i18next、react-i18next、vitest
向いている人: ローカルファイルで Markdown を管理し、クラウドに依存せず執筆したい人に向いています。Ollama などのローカル LLM を執筆補助や翻訳に手軽に使いたい人にも適しています。また、Windows / Linux（ARM64 含む）で軽量な二画面エディタを探している人や、Electron・React・WebLLM を組み合わせたデスクトップ AI アプリの実装を学びたい開発者にも参考になります。

テーマ 2. RunanywhereAI/on-device-browser-agent

https://github.com/RunanywhereAI/on-device-browser-agent

★ 305 ／ フォーク 27 ／ 言語 TypeScript ／ 作成 2026-01-18 ／ Apache-2.0
該当テーマ: WebLLM

Nanobrowser をフォークし、AI モデルを拡張機能内で動かすローカル完結型の Chrome 向け Web エージェント

概要

RA Browser Use は、自然言語で指示したタスクをもとに、Web ページの遷移・クリック・入力・情報抽出を自動で行う Chrome 拡張です。マルチエージェント型のブラウザ自動化拡張 Nanobrowser を公開フォークしたもので、モデルの実行場所をクラウドから拡張機能の内部へ移しました。推論には RunAnywhere SDK（WebAssembly + WebGPU）を使うため、API キーは不要で、既定の構成ではページ内容がブラウザの外に送られません。プライバシーを重視する人や、従量課金なしで手元の環境で試したい開発者に向いています。現在は初期開発段階で、正式リリース前です。

要点

- Nanobrowser の公開フォーク。上流のコミット履歴をそのまま保持
- 推論は拡張機能内で実行（RunAnywhere、WebAssembly + WebGPU）
- API キー不要。ハードウェアに合わせてモデルを自動選択
- クラウド LLM プロバイダーも任意で利用できる
- 初期開発段階で、対応ブラウザは Chrome と Edge のみ

何をするものか

RA Browser Use は Chrome 上で動く AI Web エージェントです。ユーザーが自然言語でタスクを伝えると、エージェントがページの移動、クリック、文字入力、情報の抽出を代わりに行います。元になった Nanobrowser は、クラウドの LLM プロバイダーや別途インストールした Ollama に接続する設計でした。このフォークは、既定でモデルを拡張機能そのものの中で動かす点が異なります。そのため既定の経路では、ページ内容もプロンプトもモデルの重みも手元のマシンから出ません。エージェントの設計、DOM のシリアライズ、拡張機能の基盤は上流の成果であると README に明記されています。

主な機能と Nanobrowser との違い

README の比較表によると、主な違いは次のとおりです。モデルの実行場所はクラウド API ではなく拡張機能の内部になり、セットアップでは API キーを用意する必要がありません。ページのデータは既定では外部に送られず、トークン単位の課金も発生しません。クラウドプロバイダーへの対応はあえて残されており、難しいタスクでは高性能な外部モデルを指定することもできます。ファイル構成を見ると、Planner や Navigator といったエージェント、ガードレール（サニタイザやパターン検出）、ファイアウォール設定、音声入力（speechToText）、チャット履歴、ブックマーク、オンボーディング画面などが含まれています。

仕組み・アーキテクチャ

pnpm と Turborepo によるモノレポ構成です。chrome-extension/src/background にエージェント本体（executor、planner、navigator、prompts など）とブラウザ制御（DOM サービス、page、context）がまとまっています。UI は pages 配下に分かれており、side-panel がチャット画面、options が設定画面、content がコンテンツスクリプトです。オンデバイス推論は pages/offscreen の offscreen document 上で動くとみられ、packages/runanywhere の ChatRunAnywhere、bridgeClient、deviceTier、modelCatalog が橋渡しやデバイス性能に応じたモデル選択を担っていると推測されます。README には、推論が Chrome の offscreen document、WebGPU、OPFS の挙動に依存すると書かれています。

技術構成

主要言語は TypeScript で、UI は React 18.3.1 と Tailwind CSS で作られています。ビルドには Vite 6、esbuild、Turborepo を使い、テストは vitest、静的解析は ESLint（airbnb-typescript）と Prettier です。husky と lint-staged でコミット前の整形も行います。推論ランタイムは vendor/runanywhere に同梱された runanywhere-web と runanywhere-web-llamacpp（v0.20.24）の tarball で、llama.cpp 系の実装を Web 向けにしたものと考えられます。リポジトリの説明文には WebLLM とありますが、README とファイル構成からは RunAnywhere SDK が使われていることが読み取れます。Node は 22.12.0 以上、pnpm は 9.15.1 が指定されています。

使い方・導入方法

Chrome Web Store にはまだ公開されていないため、ソースからビルドします。.nvmrc で指定された Node と pnpm を用意し、nvm use、pnpm install、pnpm build を順に実行します。開発中は pnpm dev でウォッチモードを使えます。engine-strict が有効なので、ツールチェーンのバージョンが合わないとインストールの段階で失敗します。ビルド後は chrome://extensions で Developer mode を有効にし、Load unpacked から dist/ を選んで読み込みます。品質チェックには pnpm type-check、pnpm lint、pnpm -F chrome-extension test が用意されています。詳しいテスト手順は docs/TESTING.md、モデル選定の背景は docs/MODELS.md にまとめられています。

注目ポイント・注意点

フォークの経緯を丁寧に開示している点が特徴です。上流のコミット履歴は squash せずにマージされ、upstream を git remote として追跡して修正を取り込む運用になっています。帰属は NOTICE に記載されています。一方で、ステータスは初期開発段階で、オンデバイス推論バックエンドはまだ接続作業中とされています。対応ブラウザは Chrome と Edge のみで、Firefox と Safari は非対応です。また README は、ブラウザ自動化には広い権限が必要であることと、クラウドプロバイダーを設定した場合はページ内容がその提供元へ送られることを、正直な注意点として挙げています。モデルの重みは初回に公開ホストからダウンロードされ、ディスクにキャッシュされます。

技術構成: TypeScript、React、Tailwind CSS、Vite、esbuild、Turborepo、pnpm、vitest、ESLint、Prettier、Chrome Extensions API、WebGPU、WebAssembly、RunAnywhere SDK (runanywhere-web / runanywhere-web-llamacpp)
向いている人: クラウドにページ内容を送らずにブラウザ操作を AI に任せたい、プライバシー重視のユーザーに向いています。API キーや従量課金なしでローカル LLM による Web エージェントを試したい開発者、WebGPU や

WebAssembly を使ったブラウザ内推論や Chrome 拡張の設計を学びたいエンジニアや学生にも参考になります。ただし正式リリース前のため、今の段階では開発者による検証や貢献が主な用途です。

テーマ 3. YuriCrystal/ai-avatar-bot

https://github.com/YuriCrystal/ai-avatar-bot

★ 204 ／ フォーク 66 ／ 言語 JavaScript ／ 作成 2026-06-09 ／ NOASSERTION
該当テーマ: WebLLM

一行の script タグで任意のサイトに埋め込める、Live2D／VRM 対応の音声 AI アバターウィジェットと運営管理画面

概要

サイト右下に Live2D や VRM の 3D キャラクターを表示し、訪問者と音声で対話しながらリップシンクで話すウィジェットです。デフォルトは純フロントエンドで、バックエンドも API キーも不要で動き、音声はブラウザ内蔵機能、回答はナレッジベース検索と任意のブラウザ内 LLM（WebLLM）で生成します。さらに Clerk と Neon Postgres を使った管理画面を備え、ナレッジベースの版管理、匿名分析、有人サポートへの引き継ぎ、見込み客収集、マルチテナントの権限管理まで扱えます。キャラクター、エンジン、コンテンツを分離し、data-* 属性で差し替えられる設計が特徴です。

要点

- embed.js を一行貼るだけで iframe ウィジェットとして埋め込める
- Live2D（2D）と VRM（3D）の両方に対応し、音量に応じたリップシンクを行う
- STT・TTS・ナレッジ検索・WebLLM をブラウザ内で完結させられる
- registerTool() でサイト操作をアバターに実行させられる
- Clerk＋Neon による管理画面で分析・有人対応・リード管理まで可能

何をするものか

Web サイトの右下に常駐するキャラクター型の音声アシスタントです。訪問者がマイクで話しかけると、アバターが内容を理解して回答し、音声で読み上げながら口の動きを同期させます。設計思想は README にある通り「エンジン（肉）＋差し替え可能な皮（キャラクターモデル）＋コンテンツ（ナレッジベース）」で、コア部分は汎用のまま、モデルや FAQ を data-* 属性で入れ替えられます。デフォルトのデモ用ナレッジはこのウィジェット自身の使い方で、アバターが自分の説明書を務める構成になっています。用途はサイト案内、カスタマーサポート、companion モードでの雑談相手などが想定されています。

主な機能

Live2D アニメーションと音量駆動のリップシンクに対応します。長い回答は文単位に分けて話し、次の文を先読みします。LLM 利用時は生成しながら話すこともできます。回答内容に応じて喜び、驚き、謝罪などの表情に切り替わります。音声入力は連続会話、途中経過の字幕表示、無音検出による自動送信、回答中の割り込みに対応し、テキスト入力も使えます。TTS はニューラル音声を使い、失敗時はブラウザ内蔵音声に自動で切り替わります。対応言語は繁体字中国語、英語、日本語、韓国語です。そのほか、setContext() によるページ文脈の受け渡し、on() によるイベント購読、companion モードでのローカル記憶（「忘記我」と言うと消去）も備えています。

仕組み・アーキテクチャ

embed.js が iframe を生成し、postMessage で親ページと通信します。あわせて window.AvatarWidget API（open、say、ask、registerTool など）を公開します。widget.html がアバター本体で、Live2D、STT、TTS、LLM、検索を担います。tool-router.js はキーワード、例文、説明、優先度をもとにツール意図をスコアリングします。候補が拮抗した場合はユーザーに選ばせ、必須パラメータは個別に聞き返し、入力スキーマで検証します。ツールの execute 関数は親ページ側にだけ残り、iframe には渡されません。バックエンドは Vercel の serverless function 群（api/ 配下）と lib/ 配下のストアで構成され、db/migrations に 001〜007 の SQL が置かれています。siteId 単位でデータを分離し、Viewer、Editor、Owner のロールで権限を管理します。

技術構成

フロントエンドは素の HTML と JavaScript です。Live2D の描画には pixi.js と pixi-live2d-display、Live2D Cubism Core を使い、3D には three.js と @pixiv/three-vrm を使います。音声認識は Chromium の webkitSpeechRecognition、ブラウザ内 LLM は @mlc-ai/web-llm（WebGPU が必要）です。管理画面では PDF.js で PDF からテキストを抽出します。バックエンドは Node 24.x 上の Vercel Functions で、依存パッケージは @clerk/backend、@neondatabase/serverless、cheerio、msedge-tts です。テストは tests/ 配下に node で直接実行するスクリプトとしてまとめられ、npm test で一括実行できます。

使い方・導入方法

導入方法は 3 段階あります。1 つ目は、widget.html、embed.js、knowledge.js の 3 ファイルを自分のサイトに置き、script タグに data-model と data-knowledge を指定する方法です。バックエンドなしで動きます。2 つ目は、デプロイ済みの embed.js を直接参照する方法です。3 つ目は、npm install の後に vercel --prod でリポジトリ全体をデプロイし、api/tts.js によるニューラル音声を有効にする方法です。3D キャラクターは .vrm ファイルをアバターにドラッグ＆ドロップするか、data-vrm で指定します。完全な管理画面を使うには、Vercel Marketplace から Clerk と Neon を導入し、.env.example に従って ADMIN_USER_IDS などを設定します。ただし .vercelignore で管理 API が除外されているため、その除外設定の調整が必要です。

注目ポイント・注意点

README はリスクと権利関係の説明が非常に丁寧です。TTS は msedge-tts を介して Microsoft Edge 読み上げの非公式エンドポイントを使っており、規約違反や停止の可能性が明記されています。本番では Azure Speech などへの切り替えが推奨されています。音声認識は Chrome の場合 Google のクラウドに送信されるため、完全なローカル処理ではありません。WebLLM は初回に約 1GB のモデルをダウンロードします。Live2D Cubism Core はプロプライエタリライセンスで、デモ用モデルも MIT ライセンスの対象外です。公開 API にはレート制限や Vercel Firewall の併用が推奨されています。なお、README では MIT とされていますが、メタデータ上のライセンスは NOASSERTION と表示されています。

技術構成: JavaScript、HTML、pixi.js、pixi-live2d-display、Live2D Cubism Core、three.js、@pixiv/three-vrm、@mlc-ai/web-llm、Web Speech API、PDF.js、Node.js 24.x、Vercel Serverless Functions、@clerk/backend、@neondatabase/serverless、Neon Postgres、cheerio、msedge-tts
向いている人: 自社サイトや個人サイトに、キャラクター付きの音声案内役や FAQ ボットを手軽に設置したい Web 開発者に向いています。VTuber 風アバターや Live2D／VRM をブラウザで動かす仕組みを学びたい学生、ナレッジ管理

や有人対応まで含む小規模なサポート基盤を Vercel 上に構築したいチームにも適しています。動作環境はデスクトップ版 Chrome が前提です。

テーマ 4. activated-intelligence/voice-chat

https://github.com/activated-intelligence/voice-chat

★ 197 ／ フォーク 25 ／ 言語 TypeScript ／ 作成 2025-12-10 ／ MIT
該当テーマ: WebLLM

音声認識・LLM・音声合成をすべてブラウザ内で動かす、ハンズフリーのAI音声チャットアプリ

概要

Whisper（音声認識）、Silero VAD（発話検出）、WebLLM上のQwen 1.5B（応答生成）、Supertonic TTS（音声合成）を組み合わせ、WebGPUでブラウザ内だけで動作するAI音声アシスタントです。APIキーもサーバーも不要で、音声データが端末の外に出ません。READMEでは内蔵LLMはあくまでデモと位置付けられています。本当の価値は配線済みの音声パイプライン（STT・VAD・TTS）にあり、約10行の変更でClaudeやGPT-4、Ollamaなど任意のLLMに差し替えられる点が特徴です。

要点

- STT・VAD・LLM・TTSがすべてブラウザ内のWebGPUで動作
- APIキー不要、音声データは端末外に送信されない
- 初回に約1GBのモデルをダウンロードし、以降はオフラインで動作
- LLMは約10行でClaude、GPT-4、Ollama等に差し替え可能
- Next.js 16とReact 19で実装、10種類の音声を同梱

何をするものか

ブラウザを開いてマイクに話しかけるだけで、AIが音声で応答するハンズフリーの音声チャットアプリです。音声認識、言語モデル、音声合成をすべてローカルのブラウザ内で実行するため、外部APIやバックエンドサーバーは使いません。プライバシーを保ったまま音声対話を試せます。HuggingFace Spaceでライブデモが公開されているので、インストールせずに動作を確認することもできます。READMEは、内蔵のQwen 1.5Bは小さなデモ用モデルにすぎず、音声パイプライン部分こそが再利用の価値を持つと明言しています。

仕組み・アーキテクチャ

処理はマイク入力から4段のパイプラインで流れます。まずSilero VADが発話区間を検出し、次にWhisperがテキストに書き起こします。続いてWebLLM上のQwenが応答を生成し、最後にSupertonic TTSがその応答を読み上げます。ファイル構成を見ると、Whisper・VADの処理はpublic/stt-worker-esm.jsのWeb Workerで、音声取得はvad-processor.jsのAudio Workletで行っています。LLM連携はhooks/use-webllm.ts、TTS連携はhooks/use-tts.tsとlib/tts.tsが担当します。lib/splitter.tsはテキストのチャンク分割を行うと説明されています。ただし、分割したテキストを逐次読み上げる詳細な実装はREADMEからは不明です。

技術構成

フレームワークはNext.js 16とReact 19で、スタイリングにはTailwind CSS v4を使っています。STTのWhisperとTTSのSupertonicは@huggingface/transformers（Transformers.js）経由で、VADのSileroはONNX

Runtime経由で、LLMのQwen 1.5Bは@mlc-ai/web-llm経由で動作します。UIにはRadix UI、class-variance-authority、lucide-react、use-stick-to-bottomなどを使い、components.jsonが置かれている点からshadcn/ui系の構成と見られます。音声は女性5種・男性5種の計10種類で、ボイス埋め込みはpublic/voicesに同梱されています。

使い方・導入方法

pnpm installで依存関係をインストールし、pnpm devで開発サーバーを起動します。その後、Chrome 113以降またはEdge 113以降でhttp://localhost:3000を開きます。動作にはWebGPU対応ブラウザ、モデル用に約4GBの空きRAM、マイクが必要です。WebGPUが使えない環境ではWASMにフォールバックし、低速ながら動作します。初回利用時にはHuggingFace CDNから、Whisper約150MB、VAD約2MB、Qwen約900MB、TTS約50MBの計約1GBをダウンロードします。ダウンロードしたモデルはIndexedDBにキャッシュされます。

LLMの差し替え

外部LLMを使う手順は2つです。まずsrc/app/api/chat/route.tsのようなAPIルートを作成します。次に、page.tsx内のhandleLLMResponse()にあるwebllm.chat()呼び出しを、fetchで/api/chatへ会話履歴をPOSTする処理に置き換えます。これでClaude、GPT-4、Geminiなどのクラウドapiや、Ollama、LM Studioといったローカル推論サーバー、OpenAI互換エンドポイントに接続できます。なお、外部LLMを使うとLLM部分は「完全ローカル」ではなくなる点に注意が必要です。

注目ポイント・注意点

音声対話に必要な要素をブラウザ内で一通り動かせる、実用的なリファレンス実装である点が魅力です。一方で、初回のダウンロード容量が約1GBと大きく、RAMも約4GB必要なため、低スペック端末やモバイルでの快適さは保証されません。対応ブラウザも実質ChromeとEdgeが前提です。内蔵のQwen 1.5Bは小型モデルなので、応答品質には限界があると考えるべきでしょう。日本語での認識や発話にどの程度対応するかはREADMEからは不明です。package.jsonのリポジトリURLやパッケージ名はirelate-voice-chatとなっており、本リポジトリ名とは異なります。

技術構成: TypeScript、Next.js 16、React 19、Tailwind CSS v4、@huggingface/transformers、@mlc-ai/web-llm、ONNX Runtime、Whisper、Silero VAD、Qwen 1.5B、Supertonic TTS、WebGPU / WASM、Radix UI、lucide-react、pnpm
向いている人: ブラウザだけで完結する音声AIアシスタントを試したい人、プライバシー重視でローカル実行の音声対話を作りたい開発者に向いています。STT・VAD・TTSの配線済みパイプラインを土台に、ClaudeやOllamaなど好みのLLMをつないだ音声UIを素早く作りたいWebエンジニアにも適しています。WebGPUやTransformers.js、WebLLMの実践例を学びたい学生にとっても良い教材です。

テーマ 5. hamzaydia/verifyfetch

https://github.com/hamzaydia/verifyfetch

★ 160 ／ フォーク 2 ／ 言語 TypeScript ／ 作成 2026-01-25 ／ Apache-2.0
該当テーマ: WebLLM

ブラウザで巨大なAIモデル等を、定数メモリでハッシュ検証しつつ途中から再開ダウンロードできるTypeScriptライブラリ

概要

VerifyFetchは、ブラウザで数GB級のAIモデルや大容量ファイルを取得する際に、SHA-256による完全性検証とレジューム可能なダウンロードを同時に実現するライブラリです。crypto.subtle.digest()がファイル全体をメモリに載せてしまう問題、回線切断時に最初からやり直しになる問題、CDNの破損やサプライチェーン攻撃に気づけない問題をまとめて解決することを狙っています。チャンク単位で到着順に検証し、IndexedDBに検証済みチャンクを保存して再開する仕組みで、メモリ使用量はファイルサイズに関係なく約2MBに保たれるとされています。Transformers.jsやWebLLM向けのドロップイン統合、CLI、Service Workerによるコード変更不要の検証も用意されています。

要点

- チャンク単位のストリーミング検証で、メモリ使用量は約2MBで一定
- IndexedDBを使い、切断時も最後に検証済みのチャンクから再開
- Transformers.jsとWebLLM向けにドロップイン統合パッケージを提供
- CLIでHugging Faceモデルのハッシュやマニフェスト JSONを一括生成
- Service Worker版なら既存のfetchを変更せず自動検証できる

何をするものか

VerifyFetchは、ブラウザ（およびNode.jsのサンプルあり）で大きなファイルを取得する際の「整合性検証」と「再開可能なダウンロード」を提供するライブラリです。READMEでは、4GBのAIモデルを読み込む途中の3.8GBで接続が切れても、3.8GBから再開しつつ全バイトを検証する、という利用シーンが示されています。基本的な使い方は verifyFetch('/model.bin', { sri: 'sha256-...' }) のようにSRI形式のハッシュを渡すだけで、ハッシュが一致しなければ例外を投げます。CDNの侵害、中間者攻撃、偶発的な破損、polyfill.io事件のようなサプライチェーン攻撃への対策を目的としています。

主な機能

コアAPIとして、通常の検証付き取得を行う verifyFetch、定数メモリでストリームとして処理しつつ最後に verified Promiseで検証結果を返す verifyFetchStream、IndexedDBにチャンクを永続化して再開可能にする verifyFetchResumable、複数CDNから sequential・race・fastest の戦略で取得する verifyFetchFromSources があります。onFail オプションでは block、warn、fallbackUrl 指定を選べます。onProgress では検証済みバイト数、総バイト数、再開フラグ、速度、ETAなどを受け取れます。さらに verifyfetch/worker の createVerifyWorker を使うと、Service Workerが *.wasm や *.onnx などのパターンに一致するfetchを自動で検証します。

仕組み・アーキテクチャ

READMEの説明によると、まずCLIでファイルのSHA-256ハッシュ（必要に応じてチャンクごとのハッシュ）を生成し、マニフェストJSONとしてアプリと一緒に配信します。VerifyFetchはファイルをチャンク単位でダウンロードし、到着したチャンクごとに検証します。破損したチャンクを検出した時点で即座に停止するため帯域を無駄にしません。接続が切れた場合はIndexedDBに保存された最後の検証済みチャンクから再開します。リポジトリ構成上、packages/core/wasm に Rust製のコード（Cargo.toml、lib.rs）があり、ハッシュ計算などにWASMを利用していると考えられますが、具体的な役割はREADMEからは不明です。

技術構成

pnpmワークスペースによるモノレポで、changesetsでリリース管理されています。packages配下に、コアの verifyfetch、Transformers.js統合の @verifyfetch/transformers、WebLLM統合の @verifyfetch/webllm、CLI の @verifyfetch/cli、人気モデルの事前計算済みハッシュを収録した @verifyfetch/manifests があります。言語はTypeScriptが中心で、ビルドにtsup、テストにVitestを使用しています。WASM部分のビルドにはRustが必要です。apps/web にはNext.js製のドキュメント兼ツールサイト（SRI生成、スキャン、プレイグラウンド）があり、examples にはNext.js、Vite、Node CLIのサンプルが含まれます。READMEでは437件のテストがあるとされています。

使い方・導入方法

コアは npm install verifyfetch で導入できます。Transformers.jsで使う場合は @verifyfetch/transformers と @huggingface/transformers をインストールし、npx @verifyfetch/cli hash-model <モデルID> でマニフェストを作成した上で、verifiedPipeline('sentiment-analysis', モデルID, { manifestUrl }) のように通常の pipeline の代わりに呼び出します。WebLLMでは createVerifiedMLCEngine にモデル名と manifestUrl を渡します。CLI には sign（SRIハッシュ生成、--chunked でチャンクハッシュ付き）、hash-model、enforce（CIでの検証）といったコマンドがあります。

注目ポイント・注意点

巨大なモデルをブラウザで扱うWeb AIの普及に合わせ、検証とレジュームを一体化した点が特徴です。一方でセキュリティモデルとして明記されているとおり、ビルドパイプライン自体が侵害されて誤ったハッシュを配信してしまう場合は防げません。その対策として verifyfetch enforce をCIで使うことが推奨されています。また事前計算済みマニフェストは現時点で Xenova/all-MiniLM-L6-v2、distilbert のSST-2モデル、Phi-3-mini など少数に限られるため、他のモデルは自分でハッシュを生成する必要があります。2026年1月作成の比較的新しいプロジェクトで、Safari等の各ブラウザでの対応状況はREADMEからは不明です。

技術構成: TypeScript、Rust (WebAssembly)、IndexedDB、Service Worker、Web Crypto / SHA-256 (SRI)、pnpm workspaces、changesets、tsup、Vitest、Next.js、Vite、Transformers.js (@huggingface/transformers)、WebLLM (@mlc-ai/web-llm)
向いている人: Transformers.jsやWebLLMなどでブラウザ上にAIモデルを読み込むWebアプリ開発者、ONNXやsafetensors、GGUF、WASMなど大容量ファイルを配信するフロントエンドエンジニア、CDN経由の配信物に対するサプライチェーン攻撃や破損対策を強化したい人に向いています。不安定な回線環境のユーザーへ大きなファイルを確実に届けたいケースにも有用です。

テーマ 6. noumena-labs/Sipp

https://github.com/noumena-labs/Sipp

★ 122 ／ フォーク 17 ／ 言語 Rust ／ 作成 2026-03-25 ／ Apache-2.0
該当テーマ: WebLLM

GGUFモデルをブラウザ上のWebGPUで動かせ、ローカル推論とクラウドゲートウェイ経由の推論を同じAPIで扱えるRust製AI推論SDK

概要

Sippは、ブラウザ、デスクトップ、エッジ、サーバーのどこでもGGUF形式のLLMを動かすための推論フレームワークです。中核の推論ランタイム「Sipp Engine」と、SippClientという共通のAPIを提供しており、WebGPUでのローカル推論とクラウドゲートウェイ経由の推論を同じ書き方で切り替えられます。ブラウザ、Node.js、Rust、Python、Swift向けのパッケージがあり、APIキーを隠すためのゲートウェイサーバーも同梱されています。READMEのベンチマークでは、WebLLMやTransformers.jsよりTTFTとデコード速度が速いとされています。

要点

- GGUFモデルをブラウザのWebGPU/WASMでローカル推論できる
- ローカル・プロバイダー・ゲートウェイを同じSippClient APIで扱える
- Browser/Node.js/Rust/Python/Swift向けのパッケージを提供
- APIキーや認証を管理するゲートウェイサーバーとAdmin UIを同梱
- 開発初期段階で、破壊的変更が入る可能性があると明記

何をするものか

Sippは、Web、デスクトップ、エッジ向けのAIアプリを作るためのオールインワンSDKです。中核にあるSipp Engineは、ブラウザ、デスクトップ、ベアメタルのクラウド環境で動くことを想定しており、起動の速さと少ないメモリ使用量を重視しています。最大の特徴は「対称なAPI」です。ブラウザ内でWebGPUを使ってGGUFモデルを動かす場合も、クラウドゲートウェイ経由でリモート推論する場合も、同じSippClientのchatやqueryで呼び出せます。エンドポイントを差し替えるだけで、処理をローカルとクラウドに振り分けられます。READMEは、ブラウザ用ランタイム、デスクトップ用のネイティブラッパー、APIキー保護用のミドルウェアがばらばらになっている問題を、一つにまとめて解決すると説明しています。

主な機能

Endpoint.localでGGUFモデルをWebGPUバックエンドに読み込んでローカル推論ができ、n_ctxなどのランタイム設定も渡せます。Endpoint.gatewayではゲートウェイのtarget、baseUrl、bearer認証を指定するだけでクラウド推論を使えます。モデルのパス、プロバイダーの認証情報、アクセスポリシー、メトリクスはゲートウェイ側が管理します。ソースツリーにはanthropic.rs、openai.rs、openai_compat.rsがあり、OpenAI、Anthropic、OpenAI互換のプロバイダーに対応していると読み取れます。Swiftではrun.tokensでトークンを逐次受け取るストリーミングの例があります。Next.js、TanStack、React+Vite向けには、SSEストリーミングや

OPFSでのモデル読み込みを含む統合ガイドが用意されています。ロードマップには、組み込みベクトルメモリ、端末上でのPIIマスキング、自動スマートルーティングが挙がっています。

仕組み・アーキテクチャ

Cargoワークスペースで構成されています。主なメンバーは、コアのcrates/sipp(パッケージ名sipp-rs)、低レベルバックエンドのcrates/sys(sipp-sys)、CLI(apps/cli)、ゲートウェイサーバー(apps/gateway-server)、ゲートウェイツールキット(lib/gateway)です。各言語向けのバインディングはbindings以下にあり、node、python、swift(UniFFI)、wasmに分かれています。コア内部にはscheduler(batch_planner、slot_scheduler)、session(kv_cache_manager、prefix_state_cache)、llama用のバッチビルダーなどのモジュールがあります。トピックにllamacppが含まれ、wasmバインディングがEmscriptenとCMakeでC++側をビルドしていることから、llama.cpp系のC++バックエンドをRustのスケジューラから制御する構成と見られますが、詳細はREADMEからは不明です。リリースビルドではfat LTOとcodegen-units=1を指定しており、マニフェストのコメントによると、これがないとwasm版のTPSが1.5〜2倍落ちるとのことです。

技術構成

主要言語はRustで、C++のバックエンドをcxxやcmake、ccでビルドしています。ブラウザ向けはEmscriptenでWASMにコンパイルし、WebGPUで推論します。各言語のバインディングには、Node.jsにnapi(napi-rs)、Pythonにpyo3、SwiftにUniFFIを使っています。その他の主な依存は、非同期処理のtokioとfutures、シリアライズのserdeとserde_json、エラー処理のthiserrorとanyhow、CLIのclapです。ゲートウェイの管理画面はReact、TypeScript、Viteで書かれ、DockerfileとNginxの設定も含まれています。ドキュメントはmdBookで生成されます。ビルドや作業の自動化はxtaskにまとめられており、sippコマンドはcargo xtaskのエイリアスです。

使い方・導入方法

新しくアプリを作る場合は、npm create @sipphq/sipp@latest my-app でブラウザアプリの雛形を生成し、npm install と npm run dev で起動します。Node.jsは20.19以上または22.12以上が必要です。既存のアプリに組み込む場合は、コードが動く場所でパッケージを選びます。ブラウザでは @sipphq/sipp、Node.jsのサーバー側では @sipphq/sipp-server を使い、後者をブラウザのバンドルに含めてはいけないと明記されています。Rustでは cargo add sipp-rs、Pythonではsipppy(リリースページのwheel)を使います。Swiftは macOS 上で sipp build swift を実行してソースからビルドします。リポジトリで開発する場合は source ./setup.sh で環境を整え、sipp doctor で確認したあと、sipp build wasm や sipp run demos serve chat などを使います。

注目ポイント・注意点

READMEのベンチマーク(RTX 3080)では、TTFTが24.3ms、デコードが77.07tok/sで、WebLLMやTransformers.jsを大きく上回る数値が示されています。ただし、モデル、プロンプト長、ブラウザのバージョンなどは記録されておらず、参考値として扱うよう注記があります。ユーザーは公開されているベンチマークサイトで自分の環境を測れます。プロジェクトは開発中で、破壊的変更が予想されるため、ミッションクリティカルな本番環境にはまだ向かない可能性があると警告されています。ゲートウェイのDockerイメージは将来提供予定で、現時点ではソースからビルドする必要があります。Swiftもソースビルドのみです。説明文にあるzero-dependencyの具体的な意味は、READMEからは不明です。

技術構成: Rust、C++、WebAssembly (Emscripten)、WebGPU、llama.cpp系バックエンド(GGUF)、CMake、cxx、napi-rs、PyO3、UniFFI (Swift)、tokio、serde、clap、React、TypeScript、Vite、Docker、mdBook
向いている人: ブラウザ内でLLMをローカル実行するWebアプリを作りたいフロントエンドエンジニアに向いています。ローカル推論とクラウド推論を同じコードで切り替えるハイブリッド構成を検討している開発者や、APIキーを隠すゲートウェイを含めて推論基盤を一式そろえたいチームにも適しています。Rust、Python、Swift(macOS/iOS)のネイティブアプリにGGUF推論を組み込みたい人も対象です。WebLLMやTransformers.jsの代わりを探している人にとっても試す価値があります。一方、開発初期段階のため、本番の基幹用途で使う場合は慎重な評価が必要です。

テーマ 7. darshi1337/apogee

https://github.com/darshi1337/apogee

★ 99 ／ フォーク 50 ／ 言語 JavaScript ／ 作成 2026-05-26 ／ MIT
該当テーマ: WebLLM

記事・動画・PDF・DOCXなどをブラウザ内のローカルAIで要約する、プライバシー重視のChrome/Firefox拡張機能

概要

Apogeeは、Webページ、YouTube/Bilibili動画、PDF、DOCX、貼り付けたテキストを、クラウドに送らずに手元の環境だけで要約するブラウザ拡張機能です。推論はWebGPU（GPU）またはWebAssembly（CPU）でブラウザ内実行するほか、自分で立てたOllamaやllama.cppのllama-serverに127.0.0.1経由で接続することもできます。サーバーで要約していたMozillaの終了プロジェクトOrbitに着想を得つつ、アカウント・APIキー・バックエンド不要のローカルファースト設計に作り直した点が特徴です。サイト別抽出、要約箇所から原文へのジャンプ、ローカル検索によるQ&A、32言語への翻訳など実用的な機能も揃っています。

要点

- 推論・検索・キャッシュをすべて端末内で実行し、クラウドAPIを使わない
- WebGPU、WebAssembly、Ollama、llama.cppの4種類の実行方式に対応
- Wikipedia、Reddit、GitHubなど多数のサイト専用抽出器を搭載
- 要約の箇条書きをクリックすると原文の該当箇所へスクロールする
- Chrome Web StoreとFirefox Add-onsで公開、MITライセンス

何をするものか

Apogeeは、ブラウザで読んでいるものを手元でAI要約するための拡張機能です。対象は一般的な記事やWebページ、選択したテキスト、YouTubeやBilibiliの動画、Redditなどのスレッド、さらにPDF、DOCX、TXT、Markdown、JSON、HTMLファイルや貼り付けたテキストまで幅広くカバーします。READMEによれば、トークナイズ、推論、検索、キャッシュはすべて端末上で行われ、ページ内容や要約がクラウドAPIに送信されることはありません。アカウント、サブスクリプション、APIキー、テレメトリ、Apogee独自のバックエンドは存在しないとされています。名前は軌道上で地球から最も遠い点を意味する言葉で、Mozilla Orbitの軌道テーマを引き継いでいます。

主な機能

記事の抽出にはReadabilityに加え、Wikipedia、GitHub、Reddit、Hacker News、Bluesky、Mastodon、Lemmy、Discourse、Stack Overflow、Lobsters、arXiv向けのサイト別抽出器を使います。動画ではタイムスタンプ付きのタイムラインを生成し、クリックで該当シーンへ移動できます。スレッドは投稿者、スコア、返信順を保った構造化Markdownに変換されます。ほかに、長文に対してローカルでパッセージを照合して答えるAsk Q&A、要約から原文への該当箇所ハイライト（Chromium系）、Chromeのサイドパネル常駐表示、ライト/ダークテーマ、独自の常設プロンプト指示、200文字までの「Focus on」キーワード指定、Helsinki-NLP Opus-MTまたはLLMによる32言語への翻訳があります。

仕組み・アーキテクチャ

ファイル構成からは、background（service-worker.js、loopbackProviders.jsなど）、content（content.js、Readability.js、サイト別extractors、highlight.js）、offscreen（offscreen.html/js）、そしてlib配下の共通ロジックに分かれていることが読み取れます。READMEでは4つのコンテキストで構成されると説明されており、詳細はdocs/ARCHITECTURE.mdにあります。libにはengines（transformersEngine、onnxWasm、ollamaClient、llamaCppClient、embeddings）、summarize（chunk、mapReduce、prompts、youtubeChapters）、retrieval（rag、passageMatch）、storage（pageCache、settings）などがあり、長文はチャンク分割とMap-Reduce方式で要約し、RAG的な検索でQ&Aを支える構成と考えられます。Ollama/llama.cppへの接続は127.0.0.1のループバックのみ許可し、それ以外のホストは拒否します。

技術構成

主要言語はJavaScriptで、Chrome/Firefox両対応のWebExtensionとして実装されています。ブラウザ内推論はトピックやファイル名からtransformers.js（transformersEngine.js、transformersLib.js）とONNXのWASM実行（onnxWasm.js）を用いていると見られ、トピックにはwebllmも含まれますが、具体的な利用範囲はREADMEからは不明です。WebGPUはChromium系で、WebAssemblyはFirefoxのデフォルトかつChromium系のオプトインのフォールバックとして使われます。PDF・DOCXの解析に使うライブラリ名はREADMEからは不明です。開発面ではESLint、Prettier、Node.js 24以上のテスト環境、GitHub ActionsによるCI、CodeQL、OpenSSF Scorecard、release-please、dependabot、gitleaksなどが整備されています。

使い方・導入方法

利用者はChrome Web StoreまたはFirefox Add-onsから拡張機能をインストールし、ページや動画、PDF、DOCXを開くか、ポップアップにテキストを貼り付け、要約形式を選んでSummarizeを押すだけです。初回はブラウザ内で選択したモデルをダウンロードし、以降はキャッシュされたモデルでオフライン動作します。より大きなモデルを使いたい場合は、Settingsからdocs/OLLAMA.mdやdocs/LLAMACPP.mdに従ってローカルのOllamaやllama-serverを設定します。開発者は apogee-extension ディレクトリで npm install と npm test を実行すると、保存済みHTMLに対してサイト抽出器のテストが走り、ブラウザやGPU、モデルのダウンロードなしで抽出器の開発に参加できます。

注目ポイント・注意点

プライバシーへの配慮が細かく、YouTubeのSponsorBlock問い合わせはk-匿名性のハッシュ接頭辞のみを送信し、設定でオフにすれば外部通信は発生しません。Gmail、Outlook、Slack、Discordなど機密性の高いサイトはディスクキャッシュから除外され、独自ドメインも追加できます。一方で、原文ハイライトはChromium系限定、Focus欄は動画やディスカッションページでは非表示といった制約があります。ブラウザ内実行ではモデルの初回ダウンロードが必要で、Firefoxでの速度低下やWebGPU非対応環境向けにトラブルシューティング文書が用意されています。first-timers-onlyやgood first issueのラベルが整備されており、OSS初参加者にも参加しやすいプロジェクトです。

技術構成: JavaScript、WebExtension (Chrome / Firefox)、WebGPU、WebAssembly、transformers.js、ONNX (WASM)、Readability、Ollama、llama.cpp (llama-server)、Helsinki-NLP Opus-MT、Node.js 24+、ESLint、Prettier、GitHub Actions
向いている人: クラウドにデータを送らずに記事や動画、PDF、社内文書などをAI要約したいプライバシー重視のユー

ザー、OllamaやllamaでローカルLLMを運用している人、WebGPUやtransformers.jsを使ったブラウザ内推論の実装例を学びたい開発者、そしてサイト別抽出器の追加などから気軽にOSSへ貢献したい学生や初学者に向いています。

テーマ 8. FMXExpress/PasClaw

https://github.com/FMXExpress/PasClaw

★ 64 ／ フォーク 16 ／ 言語 Pascal ／ 作成 2026-05-26 ／ MIT

該当テーマ: WebLLM

Delphi/Free PascalのObject Pascalで書かれた、単一バイナリで動く軽量な個人向けAIエージェントフレームワーク

概要

PasClawは、picoclawに触発されてDelphi/FPCへ移植・拡張されたAIエージェントで、CLIアシスタント、ツール呼び出し、MCP連携、OpenAI互換HTTPゲートウェイ、組み込みWeb UI、cron、スキル、チャット連携などを備えます。ネイティブビルドでは約4〜5MBの単一バイナリになり、実行時の依存はlibsqlite3とOpenSSLのみです。Anthropic、OpenAI、Geminiをはじめ約26種類のLLMプロバイダに対応し、フォールバックチェーンや自動ルーターでコストと可用性を調整できます。TPasClawAgentをTComponentとしてDelphiアプリに埋め込める点が、Pascal開発者にとって大きな特徴です。

要点

- Object Pascal製で、FPCとDelphiの両方からビルドできる
- 約26のLLMプロバイダと、フォールバック・自動ルーティングに対応
- MCPはクライアントとしてもサーバーとしても利用可能
- SQLite FTS5とローカルベクトル検索によるメモリ機能とRAGを搭載
- DockerイメージやTComponentとして手軽に導入できる

何をするものか

PasClawは、Delphi Object Pascalで書かれた超軽量の個人向けAIエージェントです。picoclawに触発されたDelphi/FPC版で、エントリポイントはsrc/pasclaw/PasClaw.dprにあり、src/cmd/以下のコマンドツリーへ処理を振り分けます。pasclaw agentによる対話チャット、pasclaw gatewayやserveによるHTTPサーバー、pasclaw cronによる定期実行、heartbeatによる能動的な定期実行など、多くのサブコマンドを持ちます。エージェントにファイル操作やシェル実行、Web検索などのツールを使わせ、作業を自律的に進めさせることが中心的な用途です。

主な機能

組み込みツールとして、read_file、write_file、edit_file、apply_patch、grep_files、shell_exec、web_search(6プロバイダ)、web_fetch、memory_search、session_search、kb_search、execute_code、spawn(サブエージェント)などを備えます。読み取り系ツールは並列にディスパッチされ、書き込み系は直列で実行されます。チャット連携はTelegram、Discord、LINE、WhatsApp、Slack、Matrix、IRC、Emailに対応しています。ほかにも、永続セッションと再開、実行中に指示を割り込ませるsteer、チェックポイントとundo、Plan/Buildモード、プロファイル、プロンプトキャッシュ、OpenTelemetryトレースなど、機能は多岐にわたります。

仕組み・アーキテクチャ

プロバイダはsrc/pkg/providers/PasClaw.Providers.Catalog.pasのカタログで管理され、新しいプロバイダは1レコード追加するだけで増やせます。Anthropic Messages、OpenAI Chat Completions、Gemini generateContentの3系統のプロトコルでSSEストリーミングに対応しています。MCPはstdioとStreamable HTTPの両方を扱い、progressive disclosureによってツール名だけをプロンプトに載せ、スキーマはtool_searchで必要なときに読み込みます。メモリはSQLite FTS5のBM25検索に加え、sqlite-vec、ONNX Runtime、MiniLMによるローカルベクトル検索をReciprocal Rank Fusionで融合します。ベクトル検索の実行環境がない場合は、自動的にFTSのみへフォールバックします。拡張用にはTPasClawHookが用意され、ツール呼び出しの前後に介入できます。

技術構成

主な言語はObject Pascalで、Free Pascal 3.2以上またはDelphi 12 / RAD Studioでビルドします。HTTPとTLSにはIndy(IndySockets)を使います。DockerfileによるとIndyはOpenSSL 1.0系を前提としているため、コンテナにはlibssl 1.0.2が同梱されています。データの保存にはSQLite(FTS5)を使います。Web UIはビルドツールを使わないvanilla ES2020の単一HTMLファイルです。デスクトップ版はFireMonkeyで作られ、ブラウザ版のbrowser-desktopはWebView2を使います。対応OSはLinux x86_64/aarch64、macOS x86_64/arm64、Windows x64で、Windows ARM64へのクロスビルドもサポートしています。container2wasmを使ったブラウザ内wasmビルドや、Replicate向けのcog定義も含まれています。

使い方・導入方法

最も手軽なのはDockerで、docker pull fmxexpress/pasclawの後にdocker run -p 8088:8088 fmxexpress/pasclawを実行すると、http://localhost:8088でWeb UIが開きます。初回起動時のウィザードで、プロバイダの選択とAPIキーの入力をブラウザ上で済ませられます。CLIで使う場合はdocker run -it fmxexpress/pasclaw agentを実行します。ネイティブビルドでは、make get-indy && makeでbuild/pasclawを生成し、pasclaw onboardで初期設定をしてからpasclaw agentで対話を始めます。Delphiアプリに組み込む場合は、TPasClawAgent.CreateしてSetProviderでキーを渡し、RegisterToolでツールを登録してからRunを呼び出します。

注目ポイント・注意点

安全対策として、パスのallowlist、シェルコマンドのdenylist、web_fetchのSSRFガード、送信者allowlist、シェル実行を使い捨てコンテナに隔離するDockerシェルバックエンド、プロンプトインジェクション(promptware)検知を備えています。ただし、シェル実行を含む強い権限をモデルに与えるため、設定の確認は欠かせません。cronは少なくとも1回の配信を保証する方式(at-least-once)なので、副作用のあるスキルは自前で重複を排除する必要があります。FPC版のtuiは履歴をメモリ上にしか保持しないなど、Delphi版と挙動が異なる部分もあります。READMEのChangelogには作成日時以降の日付が並んでいますが、各機能が実際にどこまで成熟しているかはREADMEからは不明です。

技術構成: Object Pascal、Free Pascal (FPC 3.2+)、Delphi 12 / RAD Studio、Indy (IndySockets)、OpenSSL、SQLite FTS5、sqlite-vec、ONNX Runtime、MiniLM、FireMonkey (FMX)、WebView2、vanilla JavaScript (ES2020)、Docker、container2wasm、Replicate cog (Python)、OpenTelemetry
向いている人: DelphiやFree Pascalで開発していて、自分のアプリにAIエージェント機能を組み込みたいエンジニア

に向いています。また、Node.jsやPythonの実行環境に依存しない軽量な単一バイナリのエージェントを、CLI、HTTPゲートウェイ、チャットボットとして自前で運用したい人にも適しています。MCP、RAG、マルチプロバイダ対応のエージェント基盤の実装例を学びたい学生や研究者の参考にもなります。

テーマ 9. LocalMode-AI/LocalMode

https://github.com/LocalMode-AI/LocalMode

★ 64 ／ フォーク 5 ／ 言語 TypeScript ／ 作成 2025-12-30 ／ MIT
該当テーマ: WebLLM

ブラウザ内だけで埋め込み・ベクトル検索・LLMチャット・音声・画像処理などを動かすローカルファーストAIツールキット

概要

LocalMode は、サーバーや API キーを使わず、ブラウザ上だけで AI 機能を動かすための TypeScript 製モノレポです。埋め込み、ベクトル DB、RAG、LLM チャット、構造化出力、エージェント、音声、画像処理を、モデルの初回ダウンロード後はオフラインで使えます。WebLLM、wllama、Transformers.js、LiteRT-LM、MediaPipe、Chrome Built-in AI といった複数の推論エンジンを共通インターフェースで切り替えられる点が特徴です。さらに React フック、Vercel AI SDK や LangChain.js 向けアダプタ、shadcn 形式の UI レジストリまでそろっており、プライバシーを重視する Web アプリを丸ごと組み立てられます。

要点

- 16 の npm パッケージで構成され、データを端末外に送らない設計
- WebGPU・WASM・ONNX など 4 種の LLM プロバイダを同じ API で扱える
- 依存ゼロのコアに HNSW ベクトル DB、RAG、エージェント機能を内蔵
- React フック 64 個と shadcn 形式の UI 部品 107 個を提供
- Vercel AI SDK・LangChain.js 互換で、既存アプリからの移行もしやすい

何をするものか

LocalMode は、AI を使う Web アプリをブラウザ内で 100% 完結させるためのパッケージ群です。README では「No servers. No API keys. Your data never leaves your device.」と打ち出しています。テキスト埋め込みとベクトル検索、LLM のストリーミング生成、Zod スキーマによる構造化出力、ツールを使うエージェント、音声認識・音声合成、物体検出や OCR などのビジョン処理まで幅広く扱えます。モデルを最初にダウンロードすればオフラインでも動作します。クラウド API の費用やレート制限がかからず、テレメトリもない点が利点として挙げられています。

主な機能

@localmode/core には次の機能が入っています。embed / embedMany による埋め込み、streamText / generateText による生成、generateObject による構造化出力、classify / extractEntities / rerank / translate / summarize といった NLP タスク、transcribe / synthesizeSpeech による音声処理、detectObjects / segmentImage などのビジョン処理です。ベクトル DB は HNSW インデックスを使い、IndexedDB への永続化、タブ間同期、SQ8 や Product Quantization による圧縮、WebGPU を使った距離計算、BM25 とのハイブリッド検索に対応します。ほかにも、チャンク分割や取り込みパイプライン、セマンティックキャッシュ、評価 SDK、Web Crypto を使った暗号化、PII 除去、差分プライバシー、改ざん検知できる監査ログを備えています。

仕組み・アーキテクチャ

構成は層状です。最下層にブラウザ API（WebGPU、WASM、IndexedDB、Web Workers、Web Crypto）があり、その上にプロバイダパッケージ、さらに依存ゼロの @localmode/core が載ります。その上に langchain・ai-sdk・devtools の各アダプタ、最上位に @localmode/react があります。設計原則として、ML フレームワークを別パッケージに分けるプロバイダパターン、クラスではなくトップレベル非同期関数を中心にした API、単一のオプションオブジェクト、{ result, usage, response } 形式の戻り値、全非同期処理での AbortSignal 対応、ミドルウェアによるラップの合成が明記されています。WebGPU が使えない環境では WASM へ、IndexedDB が使えない環境ではメモリへ、自動でフォールバックします。

技術構成とプロバイダ

言語は TypeScript で、pnpm と changesets を使ったモノレポです。LLM のプロバイダは 4 種類あります。WebGPU で最速の WebLLM（厳選 32 モデル）、llama.cpp の WASM 版で GGUF モデルを扱う wllama（厳選 30 モデルに加え、HuggingFace の 160K 以上のモデル）、ONNX ベースの Transformers.js、Google の LiteRT-LM です。このほか、手・姿勢・顔のトラッキングを担う MediaPipe と、Gemini Nano を使う Chrome Built-in AI も用意されています。ストレージアダプタは Dexie.js、idb、localForage から選べ、PDF 抽出には PDF.js を使います。ドキュメントサイト apps/docs は Next.js と Fumadocs で構築されているようです。

使い方・導入方法

たとえば意味検索なら、pnpm install @localmode/core @localmode/transformers でインストールします。そのうえで transformers.embedding('Xenova/bge-small-en-v1.5') でモデルを作成し、createVectorDB でベクトル DB を用意して、chunk、embedMany、db.addMany、db.search の順に呼び出します。LLM を使うときは、webllm.languageModel などでモデルを選び、streamText に渡します。React では useChat などのフックを使います。UI 部品は shadcn CLI で導入でき、components.json にレジストリを登録したうえで npx shadcn@latest add @localmode/ui/conversation のようにコピーします。動作するデモは localmode.ai/blocks で確認でき、詳しいドキュメントは localmode.dev にあります。

注目ポイント・注意点

注目点は、複数のブラウザ推論エンジンを 1 つのインターフェースにまとめていることです。さらに、エージェントのツール実行に人間の承認を挟む仕組みや、MLPerf-Client 互換の指標でベンチマークを取る @localmode/bench まで含んでいます。一方で注意点もあります。WebLLM と LiteRT は WebGPU 必須のため、Chrome / Edge 113 以降が前提です。Chrome AI は Chrome と Edge でしか使えません。LiteRT は「early preview」と明記されています。devtools のウィジェット UI は v3.0.0 で削除されました。また、モデルの初回ダウンロード量や端末の性能が実際の使い勝手を大きく左右すると考えられますが、具体的な数値は README からは不明です。スター数はまだ少なく、比較的新しいプロジェクトです。

技術構成: TypeScript、React、Next.js、WebGPU、WebAssembly、Transformers.js (ONNX Runtime)、WebLLM、wllama / llama.cpp、LiteRT-LM、MediaPipe、Chrome Built-in AI (Gemini Nano)、Vercel AI SDK、LangChain.js、shadcn/ui、Zod、PDF.js、Dexie.js、idb、localForage、IndexedDB、Fumadocs、pnpm、changesets
向いている人: ユーザーのデータを外部に送らずに AI 機能を提供したい Web フロントエンドエンジニアや React 開発者に向いています。オフラインで動く PWA、GDPR などの規制を意識したアプリ、ブラウザ拡張機能に AI を組み

込みたい人、API 費用をかけずに RAG やチャットを試したい学生や個人開発者にも適しています。Vercel AI SDK や LangChain.js で作った既存アプリをローカル推論へ移したいチームにも有用です。

テーマ 10. luskB/MCP-WebLLM

https://github.com/luskB/MCP-WebLLM

★ 57 ／ フォーク 10 ／ 言語 JavaScript ／ 作成 2026-03-05 ／ MIT
該当テーマ: WebLLM

ChatGPTやDeepSeekなど網頁版AIチャットからローカルのMCPツールを呼び出せるようにするChrome/Edge拡張機能

概要

MCP Multi Bridge は、ChatGPT、Gemini、DeepSeek、Kimi など14以上のWeb版AIアシスタントの画面に、ローカルで動くMCP（Model Context Protocol）サーバーのツールを接続するブラウザ拡張です。ネイティブクライアントを使わずに、ブラウザ上のAIにローカルファイルの読み取りやDB問い合わせ、検索などをさせたい人向けに作られています。React/TypeScript系の既存プロジェクトとは違い、フレームワークもビルド工程も持たない純粋なVanilla JSで書かれており、クローンしてそのまま読み込めます。SSE、Streamable HTTPに加えて、Chrome Native Messaging経由のStdio直結にも対応しているのが特徴です。

要点

- Web版AI 14以上のサイトにMCPツール用サイドバーを注入する
- 依存ゼロのVanilla JSで書かれ、Manifest V3に対応、ビルド不要
- SSE・Streamable HTTP・Stdioの3種類の転送方式に対応
- AIが出力したjsonl形式のツール呼び出しを検出し、自動で実行・結果を返送できる
- Agentモード用のプロンプトを内蔵し、AIが自律的に繰り返しツールを呼び出せる

何をするものか

Web版AIチャットとローカル環境の間をつなぐChrome/Edge拡張機能です。対応サイトは ChatGPT、Google Gemini、Google AI Studio、Grok、Perplexity、GitHub Copilot、DeepSeek、OpenRouter、Mistral、T3 Chat、Kimi、通義千問、智譜清言、豆包です。これらのサイトでは、ローカルのMCPサーバーが提供するツールを呼び出せるようになります。仕組みとしては、まずツール定義と呼び出し規約をまとめたシステムプロンプト（mcp-tools.md）をAIに渡します。AIがその書式どおりにツール呼び出しを出力すると、拡張がそれを検出して実行し、結果を会話に戻します。プロダクトのコンセプトとUIの考え方は MCP-SuperAssistant を参考にしたと README に明記されています。

主な機能

ページ右側の MCP ボタンからサイドバーを開くと、ツール・呼び出し・設定の3タブが使えます。ツールタブでは、接続中サーバーのツールが一覧表示され、プロンプトを.mdファイルとして添付・ダウンロード・コピーできます。呼び出しタブには、検出したツール呼び出しごとにカードが作られ、パラメータ、状態、結果のプレビューが表示されます。AIの出力中にもインラインの実行ボタンが挿入されます。設定タブでは、自動実行、結果注入後の自動送信、貼り付けのインターセプト、注入待ち時間（既定4秒）、送信前の待ち時間（既定8秒）を変更できます。ほかにも、複数サーバーへの同時接続とツール名による自動ルーティング、複数呼び出しの一括注入、ダークモード、サーバー設定のその場での編集に対応しています。

仕組み・アーキテクチャ

background.js は Service Worker として動作し、SSEParser、MCPConnection、ServerManager の3層でMCPサーバーとの接続とJSON-RPCの中継を担います。content.js は各AIページに注入されます。Shadow DOMでスタイルを分離したサイドバーを作り、MutationObserverでDOMを監視して、jsonlコードブロック（function_call_start など）を parseToolCalls() で解析します。Stdio転送の流れは次のとおりです。chrome.runtime.connectNative() で Node.js 製の native-host.js に接続し、native-host.js が子プロセスとしてMCPサーバーを起動して、stdin/stdout でメッセージを双方向に転送します。また、dragDropListener.js は Gemini 専用で、ページのメインワールドでドラッグ&ドロップイベントを組み立ててファイルアップロードを再現します。

技術構成

拡張本体は、フレームワークもバンドラーも使わない Vanilla JavaScript とネイティブCSSで構成されています。Manifest V3 の拡張で、要求する権限は storage、nativeMessaging、host_permissions: <all_urls> です。<all_urls> はローカルMCPサーバーへのクロスオリジンfetchに使うと説明されています。Stdio転送のホスト側は Node.js（16以上）で動く native-host.js で、install-host.bat が Native Messaging Host のマニフェスト（com.mcp.bridge.json）を生成し、Chrome と Edge のレジストリに登録します。設定は chrome.storage.local に保存されます。Service Worker が停止しないよう、chrome.runtime.connect() のポート接続で生存を維持しています。

使い方・導入方法

リポジトリをクローンし、chrome://extensions/ で開発者モードをオンにして「パッケージ化されていない拡張機能を読み込む」からプロジェクトのルートを選びます。対応ブラウザは Chrome または Edge のバージョン109以上です。SSEで使う場合は、たとえば npx -y supergateway --stdio "npx -y @modelcontextprotocol/server-filesystem 作業ディレクトリ" --port 3006 でサーバーを起動します。そのあとポップアップで URL http://localhost:3006/sse を登録します。Stdioで使う場合は、拡張IDを引数にして install-host.bat を実行し、ブラウザを再起動します。続いて Command に npx、Args に -y @modelcontextprotocol/server-filesystem パス を入力します。Args にコマンド名を含めないよう注意してください。最後にAIサイトのサイドバーで「附加 .md」を押してプロンプトをAIに送れば準備完了です。

注目ポイント・注意点

Stdio のセットアップ手順として README に載っているのは Windows 用の install-host.bat だけです。macOS と Linux でのセットアップ方法は README からは分かりません。ツール呼び出しは、AIが指定のjsonl形式に従うことが前提です。そのため、最初にプロンプトファイルを読ませる必要があります。結果のファイル注入は、ファイル入力、ドラッグ&ドロップ、貼り付けの順に試し、すべて失敗した場合はテキスト貼り付けに切り替わります。サイト側のUIが変わると動かなくなる可能性があります。自動実行と自動送信を両方オンにすると、AIがローカルツールを人の確認なしに繰り返し呼び出します。ファイル操作などを扱う場合は、手動確認を有効にしておくのが安全です。失敗時は最大5回まで再試行し、間隔はn秒の線形バックオフ、タイムアウトは30秒です。なお README には、OPENCODE と Claude Opus 4.6 を使って開発したと書かれています。

技術構成: JavaScript (Vanilla JS)、CSS、Chrome Extension Manifest V3、Chrome Native Messaging、Node.js、Shadow DOM、MutationObserver、Model Context Protocol (SSE / Streamable HTTP / Stdio)、JSON-RPC、Windows バッチスクリプト
向いている人: ChatGPT、Gemini、DeepSeek、Kimi などのWeb版AIを普段使っていて、ローカルのファイル、データベース、検索ツールをMCP経由でつなぎたいエンジニアや学生に向いています。ビルド工程のない素朴なコードで、Chrome拡張やMCPの転送方式（SSE / Streamable HTTP / Native Messaging）の実装を学びたい人の教材にもなります。ただし Stdio の導入手順は Windows 向けしか記載されていない点に注意が必要です。

テーマ 11. webllm/renderify

https://github.com/webllm/renderify

★ 33 ／ フォーク 2 ／ 言語 TypeScript ／ 作成 2026-02-11 ／ MIT
該当テーマ: WebLLM

LLMが生成したJSX/TSXをビルド不要でブラウザ上で直接実行・描画するランタイム優先のUIエンジン

概要

Renderifyは、LLMが出力したJSX/TSXや構造化されたRuntimePlan(JSON)を、バックエンドのビルドサーバーやデプロイ工程を挟まずにブラウザ上で即座に対話的なUIとして描画するためのTypeScript製ライブラリです。@babel/standaloneによるインライン変換と、JSPM/CDN経由のモジュール解決およびmoduleManifestによるバージョン固定で、バンドラーを使わない実行を実現しています。既存のJSONスキーマ型レンダラー(A2UI、json-render)のように定義済みコンポーネントに縛られず、v0やSandpackのような重いビルド基盤も不要な点を差別化要素としています。実行前のセキュリティポリシー検査、ストリーミング描画、MCP Appsアダプターなど、LLMチャットやエージェント基盤への組み込みを意識した機能が揃っています。

要点

- LLM生成のJSX/TSXをビルド工程なしでブラウザ実行
- JSPM経由でnpmパッケージを解決しmoduleManifestで固定
- 実行前に4段階のセキュリティプロファイルで検査
- LLMは任意で、レンダラー単体としても利用可能
- MCP Appsアダプターやストリーミング描画にも対応

何をするものか

Renderifyは「LLMはUIコードを書けるが、それを直接ブラウザで見せる良い方法がない」という課題に取り組むランタイムです。LLMの出力(JSX/TSXのコードブロック、または構造化JSONのRuntimePlan)を受け取り、CodeGenで解析・正規化し、セキュリティポリシー検査を経てから、Babelによる変換とJSPMによるモジュール解決を行ってブラウザに描画します。READMEでは、v0/Bolt.newはビルドバックエンドが必要、Streamlit/Gradioはサーバー描画、JSONスキーマ型レンダラーはスキーマ外を表現できない、Sandpack/WebContainersは重量級、と既存手法を比較し、軽量でセキュリティ統制された実行環境という位置づけを示しています。

主な機能

ゼロビルド描画に加え、bare import(例: date-fns)をJSPMで自動解決して最新版をmoduleManifestに固定するauto-pin-latest、strict・balanced・trusted・relaxedの4つのセキュリティプロファイル、構造化RuntimePlanと生のTSX/JSXの二系統入力をサポートします。renderPromptStreamはllm-delta、preview、finalのチャンクを出し、チャットUIで段階的に描画できます。beforeLLMやafterCodeGen、beforeRenderなど10個のフックポイントによるプラグイン拡張、createInteractiveSessionによるイベント→状態→再描画

ループ、CSS/JSONインポートのプロキシ化、react系パッケージをpreact/compatへ橋渡しする互換レイヤー(recharts、@mui/materialなど)も備えています。

仕組み・アーキテクチャ

パイプラインは Prompt → LLM Interpreter → Code Generator → Runtime Plan(IR) → Security Policy Checker → Runtime Executor → UI Renderer という流れで、Executorから JSPM Module Loader(JSPM CDN / SystemJS)を呼び出します。ブラウザ側ではfetchでモジュールを取得し、importを書き換えてblob URL化することでbare specifierを解決するモジュールグラフ実体化を行います。import指定子の抽出にはes-module-lexerを使い、正規表現の脆さを避けています。実行前にはimports、コンポーネントモジュール、ソースimportを調べるプリフライトがあり、リトライ・タイムアウト・複数CDNフォールバックにも対応します。ソースのサンドボックスとしてsandbox-worker / sandbox-iframe / sandbox-shadowrealmがありますが、READMEによれば三つとも実体は終了可能なWorker境界です。

技術構成

pnpmワークスペースとturboによるモノレポで、packages配下にrenderify(トップレベルSDK)、@renderify/ir(Plan/Node/State等のIR定義)、@renderify/runtime(実行エンジンとJSPMローダー)、@renderify/security(ポリシー)、@renderify/mcp-app(MCP Appsアダプター)、@renderify/core(旧来の互換層)、@renderify/llm(OpenAI、OpenAI Codex、Anthropic、Google、Ollama、LM Studioプロバイダ)、@renderify/cli(CLIとPlayground)が並びます。ビルドはtsup、Lint/FormatはBiome、テストはtsx --testとPlaywright、ベンチマークはtinybench、リリース管理はChangesetsです。ドキュメントサイトはapps/websiteにNext.jsとFumadocs構成で置かれています。Node.js 22以上が必要です。

使い方・導入方法

SDKとして使う場合は pnpm add renderify でインストールし、renderTrustedPlanInBrowser(plan, { target: "#mount" }) や renderPlanInBrowser にRuntimePlanを渡すだけで描画できます。ビルドツールなしの素のHTMLで使う場合は、事前に@babel/standaloneを読み込む必要があります。本番環境ではautoPinLatestModuleManifest: falseとし、moduleManifestにresolvedUrlとversionを明示するmanifest-only運用が推奨されています。リポジトリ内では pnpm cli -- run や plan、probe-plan、render-plan などのCLIコマンド、pnpm playground によるローカルPlaygroundが使え、RENDERIFY_LLM_PROVIDERなどの環境変数でLLMプロバイダやセキュリティ設定を切り替えられます。インストール不要の静的Playgroundも公開されています。

注目ポイント・注意点

bundlelessとはnpmバンドラーとの完全互換を意味せず、preact/reactブリッジやrechartsは保証、純粋なブラウザESMパッケージはベストエフォート、Node.js組み込みモジュールは確定的に拒否という段階的な互換契約が明示されています。またREADMEは、ソース走査やサンドボックスは多層防御であって、任意の悪意あるJavaScriptの安全を保証するものではないと明言しています。strictとbalancedではランタイムソースモジュールが拒否され、isolated-vmプロファイルは将来用の予約で現状は失敗終了します。MCP Appsアダプターはソース・ネットワーク・タイマー等を一切許さない狭い契約です。作成が2026年2月と新しく、スター数も少ないため、成熟度はREADMEからは判断しにくい点に留意が必要です。

技術構成: TypeScript、Preact、@babel/standalone、JSPM / SystemJS、es-module-lexer、@modelcontextprotocol/sdk、pnpm、turbo、tsup、Biome、Playwright、tinybench、Changesets、zod、Next.js / Fumadocs(ドキュメントサイト)
向いている人: LLMチャットやAIエージェント基盤で、モデル出力から動的なダッシュボード・フォーム・可視化UIを即座に描画したい開発者に向いています。ローコード/ノーコードのAIバックエンド、動的フォームや承認フロー、プロンプトから数秒でUIを試すプロトタイピング、MCP Appsでインタラクティブなビューを MCPツールから提供したいケースにも適しています。LLMを使わずに、自前のバックエンドが生成したRuntimePlanを安全に描画するレンダラーとしての利用も想定されています。

テーマ 12. Kriswd/webllm-gateway

https://github.com/Kriswd/webllm-gateway

★ 32 ／ フォーク 7 ／ 言語 Python ／ 作成 2026-04-28 ／ MIT
該当テーマ: WebLLM

Qwen や DeepSeek の Web 版チャットを、OpenAI／Anthropic 互換でツール呼び出し可能な API に変換するゲートウェイです。

概要

WebLLM Gateway は、ブラウザでログインして使う Qwen Web や DeepSeek Web などの Web 版モデルを、OpenAI 互換の /v1/chat/completions と Anthropic 互換の /v1/messages で使えるようにするローカル API ゲートウェイです。OpenClaw、Hermes、Claude Code、Codex などのエージェント系クライアントから、ネイティブ API と同じ感覚で Web アカウントのモデルを呼び出せます。単純な中継ではなく、ToolBridgeV2 によって Web モデルの自然言語出力を検証し、標準の tool_calls／tool_use 構造に変換する点が特徴です。ツールの実行自体はクライアント側に任せ、ゲートウェイはプロトコル変換、ログイン管理、モデル目録の管理に役割を絞っています。

要点

- Web版のQwen/DeepSeekをOpenAI・Anthropic互換APIとして公開する
- ToolBridgeV2がWebモデル出力を標準tool_callsへ変換・検証する
- ブラウザでログインするだけで、cookieなどを手動で取り出す必要がない
- ゲートウェイ自体はローカルツールを実行せず、権限管理はクライアント側に残る
- 学習・研究用途向け。各サービスの利用規約は利用者自身が確認する必要がある

何をするものか

Web ブラウザでログインして使うタイプの LLM（Qwen Web、DeepSeek Web など）を、標準的な API として使えるようにするゲートウェイです。README では「不安定な Web 対話を、検証可能・接続可能・ツール呼び出し可能な標準 API に整える」ことを核心価値に掲げています。使い方は、Web アカウントでログインし、利用可能なモデルを自動検出したうえで、API アドレス、API Key、モデル ID を OpenClaw や Hermes などのクライアントに設定するだけです。想定用途は資料整理、情報検索、簡単な自動化といった軽〜中程度のタスクで、API トークン消費の節約や低コストでの試行錯誤を狙っています。一方で、複雑なエンジニアリングや長時間の多ターンプログラミングは、現時点で主な用途としていないと明記されています。

主な機能

API は OpenAI 互換の /v1/chat/completions と、Anthropic 互換の /v1/messages および /v1/messages/count_tokens を提供します。ストリーミング、x-api-key と Bearer の両方の認証、近似トークン計数にも対応しています。統合コンソールからは、Web ログインの認可、実際のログイン状態の検出、利用可能モデルの一覧表示、接続設定のコピー、ローカルトークンの再生成が行えます。モデル目録では、Qwen Web の場合ログイン後に実際の目録を読み込み、qwen-web/qwen3.7-max などのモデルを提供します。古いモデル ID は、現在も使える Web モデルにマッピングされます。ほかに、WebAI2API 経由で ChatGPT を使う /v1/images/

generations（gpt-image-2 推奨）による画像生成もあります。実経路で検証されていないモデルは、目録から除外される方針です。

仕組み・アーキテクチャ（ToolBridgeV2）

ツール呼び出しの中核は、デフォルトで strict モードとして動く ToolBridgeV2 です。クライアントから届いた tools は Web モデルにそのまま渡さず、厳格な tool_json プロトコルの指示に変換してプロンプトに注入します。モデルがツールを使う場合は、fenced 形式の tool_json ブロック（calls 配列に id、name、input を含む形）を 1 つだけ出力させます。ゲートウェイは、ツール名、引数、ID の重複、呼び出し数を検証してから、標準の tool_calls／tool_use に変換します。JSON が壊れていれば一度だけ自動修復を試み、それでも失敗した場合は x-webai-tool-bridge-error ヘッダで診断情報を返します。tool_result は観察テキストに変換し、長すぎる結果は圧縮します。さらに、promptMaxChars（デフォルト 32000 字）を超える入力では、中間部分を圧縮します。

技術構成

バックエンドは Python 製で、requirements.txt には fastapi、uvicorn、httpx、pytest、playwright が並んでいます。webai_gateway パッケージの中には、openai_api.py、anthropic_api.py、tool_bridge.py、qwen_web.py、deepseek_web.py、web_auth.py などのモジュールがあります。管理画面の webui は Vue（.vue ファイル）と Vite で作られ、pnpm でビルドします。Dockerfile は、Node 22 で WebUI をビルドし、Python 3.12 の slim イメージで実行する 2 段構成です。DeepSeek は外部の ds2api サイドカー（デフォルトは 127.0.0.1:9331）経由、ChatGPT などは WebAI2API（デフォルトは 127.0.0.1:8500）経由で接続します。これらはソースに同梱されないオプションの runtime で、入っていなくてもゲートウェイ本体は起動します。テストには、ds2api を oracle とした parity テストや、多数のツール呼び出し replay fixture が含まれています。

使い方・導入方法

Windows では start_webai_gateway.bat をダブルクリックして起動し、http://127.0.0.1:8610/ の統合コンソールを開きます。停止するときは stop_webai_gateway.bat を使います。OS をまたいで同じ手順で動かしたい場合は、docker compose up -d --build で起動できます。この場合、設定、認証情報、ログは webai-gateway-data ボリュームに保存されます。コンソールの「網頁登録向導」で認可用ブラウザを開いてログインし、モデルを検出したら、base_url を http://127.0.0.1:8610/v1、api_key を local-dev-key、model を qwen-web/qwen3.7-max のように設定します。NAS、Docker、リモート環境では、手元の PC で --remote-debugging-port=9222 を付けて Chrome／Edge を起動し、その CDP アドレスを指定してログイン状態を検出する方法が推奨されています。

注目ポイント・注意点

ゲートウェイの責任範囲が明確に区切られている点が設計上の特徴です。Skill の展開、MCP 呼び出し、ファイル編集、ターミナルコマンド、権限確認はすべて下流クライアントが担い、ゲートウェイはそれらを実行しません。特定のクライアントやモデル向けの特別処理は避け、observationPolicy などの汎用設定で調整する方針も示されています。一方で、Web アカウントを API 化する性質上、各サービスの利用規約やアカウント規約に抵触するおそれがあります。README の免責事項は、学習・研究・ローカルでの技術検証に用途を限定し、有料制限の回避や不正利用を禁じています。アカウントに関するリスクは利用者自身が負うとされている

ため、導入前に各サービスの規約を確認することが不可欠です。また、Qwen Web 直結は現時点ではテキストのみに対応しており、マルチモーダル入力は拒否されます。

技術構成: Python、FastAPI、uvicorn、httpx、pytest、Playwright、Vue、Vite、pnpm、Node.js 22、Docker / Docker Compose、Chrome DevTools Protocol (CDP)、ds2api（外部サイドカー）、WebAI2API（外部 runtime）
向いている人: OpenClaw、Hermes、Claude Code、Codex などのエージェントクライアントで、Qwen や DeepSeek の Web 版モデルを試したい開発者に向いています。Web 版モデルを OpenAI／Anthropic 互換のツール呼び出し経路に組み込む方法を研究・検証したいエンジニアや学生にも適しています。資料整理や簡単な自動化といった軽〜中程度の用途で、API コストを抑えて試行錯誤したい人向けです。ただし、各サービスの利用規約を自分で確認し、学習・研究目的の範囲で使うことが前提です。

テーマ 13. salpbes/GomeraX

https://github.com/salpbes/GomeraX

★ 28 ／ フォーク 11 ／ 言語 TypeScript ／ 作成 2025-11-02 ／ MIT
該当テーマ: WebLLM

ブラウザ内で動くローカルLLMのBIMアシスタントを備えた、Open BIM Componentsベースの実験的IFCビューア

概要

GomeraXは、建築分野の標準データ形式であるIFCファイルをブラウザで閲覧・分析するための実験的なビューアです。That Open CompanyのOpen BIM Components（OBC）とThree.jsを基盤に、断面表示、計測、平面図ビュー、一人称ウォークスルー、ダッシュボード分析など、BIM閲覧に必要な機能をモジュール単位で多数実装しています。最大の特徴は、WebLLMとQwen3を使ってLLMをブラウザ内だけで動かし、「すべてのドアを表示して」のような自然言語でモデルを操作できる点です。さらに、大規模モデル向けに実験的なWebGPUレンダリングモードも用意されています。

要点

- IFCとFragments形式を読み込めるブラウザ向けBIMビューア
- WebLLM＋Qwen3による完全ローカルの自然言語BIMアシスタント
- LOD・GPUピッキングなどを備えた実験的WebGPUモード
- 断面・計測・平面図・FPS歩行・クラスタリングなど多機能
- npm install と npm run dev ですぐに試せるMITライセンス

何をするものか

GomeraXは、建築・建設分野で使われるIFC（Industry Foundation Classes）ファイルをWebブラウザ上で3D表示し、要素のプロパティを確認・分析するためのビューアです。Open BIM Components（@thatopen/components など）を土台にしており、IFCを高速に扱えるFragments形式へ変換して読み込むこともできます。READMEでは自らを「実験的」なデモプロジェクトと位置づけており、表示結果やAIの出力を設計・施工の判断に使う前には、認証済みの業界標準ソフトで検証するよう免責事項で明記しています。

主な機能

基本機能として、IFC・Fragmentsの読み込みのほか、Fragments、glTF、GLB、USDZ、PNGスクリーンショット、プロパティJSONへのエクスポートに対応しています。閲覧系では、ViewCubeによる視点切替、X/Y/Zプリセットやダブルクリックで作る断面、長さ・面積・体積の計測、2D平面図ビュー、重力と衝突判定付きの一人称ウォークモードを備えます。分析系では、IFCツリーとプロパティパネル、Excel風のプロパティテーブル、Power BI風のModel Dashboard、複数条件で絞り込めるData Slicerがあります。ほかに、要素をIFCカテゴリごとに物理的に並べ替えて色分けするElement Clusteringも搭載しています。

ローカルAIアシスタント

AIアシスタントは、WebLLM（@mlc-ai/web-llm）を使ってQwen3をブラウザ内で動かす、プライバシー重視の設計です。使い方の例として、READMEには次のような自然言語の指示が挙げられています。要素の選択・非表示・分離（"show me all doors"）、代名詞を使った続けての指示（"hide them"）、要素数の問い合わせ（"how many windows are there?"）、カメラ操作、階名での移動（"go to Level 2"）です。実装は、src/modules/core/ai 以下で意図検出（AIIntentEngine、IntentDetector）、ルールベース応答（AIRuleEngine）、会話コンテキスト管理、ツール実行（ToolRegistry、ToolExecutor）に分かれています。agent/strategies には FunctionCallingStrategy と HybridStrategy があり、LLMとルールを組み合わせた構成とみられますが、詳細な動作はREADMEからは不明です。

仕組み・アーキテクチャ

コードは、IFCViewer.ts をオーケストレーターとしたモジュール構成です。core にはローダー、プロパティ、AI、性能監視、webgl にはWorldManager・Clipper・Measurement・FloorPlan・FirstPersonControlsなどの描画機能、webgpu には実験的レンダラーと各種マネージャ、ui にはツールバーやダッシュボードのUIがまとまっています。座標については、FragmentsManagerで COORDINATE_TO_ORIGIN = false を指定して元のプロジェクト座標を保持し、意匠・構造・設備など複数モデルを自動で位置合わせします。原点から100km以上離れたモデルは検出して警告を出します。ファイル一覧には、READMEで触れられていない ClashDetectionModule.ts と ClashUIManager.ts もありますが、干渉チェックの仕様はREADMEからは不明です。

技術構成と導入方法

TypeScriptとViteで構築されています。主な依存ライブラリは、@thatopen/components、@thatopen/components-front、@thatopen/fragments、@thatopen/ui、@thatopen/ui-obc、three、web-ifc、@mlc-ai/web-llm、@huggingface/transformers、chart.js、stats.jsです。導入にはNode.js v18以上が推奨されています。npm install で依存関係を入れ、npm run dev を実行して http://localhost:3000 を開けば起動します。本番ビルドは npm run build（出力先は dist）、確認は npm run preview です。READMEには非開発者向けの丁寧な手順書と、依存関係の解決に失敗した場合に npm install --legacy-peer-deps を使うといったトラブルシューティングも載っています。推奨ブラウザはChromeまたはEdgeです。

注目ポイント・注意点

ブラウザだけで完結するLLMをBIM操作に組み込んだ点は、クラウドにモデルデータを送らずに済むという意味で興味深い試みです。WebGPUモードでは、LOD、GPUカラーピッキング、マルチパスのアウトライン、フォグ、チャンク単位のシーン再構築、適応的な品質調整などが実装されています。一方で注意点もあります。WebGPU機能は実験的で、ハードウェアによって動作が安定しない可能性があると明記されています。LLMについても、誤った情報（ハルシネーション）を出す可能性があるとされています。ローカルLLMの実行にはモデルのダウンロードやGPU性能が必要になると考えられますが、必要スペックはREADMEからは不明です。なお、READMEの技術一覧の版番号と package.json の版番号には一部食い違いがあります。

技術構成: TypeScript、Vite、Three.js、@thatopen/components、@thatopen/components-front、@thatopen/fragments、@thatopen/ui、@thatopen/ui-obc、web-ifc、@mlc-ai/web-llm (Qwen3)、@huggingface/transformers、chart.js、stats.js、WebGL、WebGPU

向いている人: BIMやIFCを扱う建築・建設分野のエンジニア、OpenBIMやThat Open Componentsを使ったWebビューアの実装例を学びたい開発者に向いています。ブラウザ内LLM（WebLLM）やWebGPUレンダリングを実アプリに組み込む手法に関心のある人や学生にも参考になります。ただし実験的プロジェクトなので、実務の判断に直接使うのではなく、検証やプロトタイピング、学習用途での利用が適しています。

テーマ 14. webllm/webblackbox

https://github.com/webllm/webblackbox

★ 15 ／ フォーク 1 ／ 言語 TypeScript ／ 作成 2026-02-12 ／ MIT
該当テーマ: WebLLM

Webアプリの操作・通信・ログ等を常時記録し、後から再生・解析できるフライトレコーダー兼タイムトラベルデバッガー

概要

WebBlackbox は、Webアプリ内で起きたユーザー操作、ナビゲーション、コンソール出力、ネットワーク通信、ストレージ、DOM、スクリーンショット、パフォーマンス情報をまとめて記録するツールです。記録した内容はポータブルな .webblackbox アーカイブに書き出され、ホスト版 Player で後からタイムライン付きで再生できます。Chrome 拡張、アプリに埋め込む SDK、解析用の Player SDK、AI 連携用の MCP サーバーまでを一つのモノレポで提供しています。「再現しにくいバグを、起きたときの状況ごとチームに渡せる」点と、curl や HAR、Playwright スクリプト、バグレポートを自動生成できる点が特徴です。

要点

- 13カテゴリ57種類のイベントを一つのアーカイブに記録
- 負荷の低い lite と CDP を使う詳細な full の2モード
- AES-GCM で暗号化できる .webblackbox 形式で共有可能
- Player から curl・HAR・Playwright・バグレポートを生成
- MCP サーバー経由で AI アシスタントによる解析に対応

何をするものか

WebBlackbox は README で「Webアプリケーションのためのフライトレコーダー兼タイムトラベルデバッガー」と説明されています。飛行機のブラックボックスのように常に記録を続けておき、問題が起きたときに何が起きたのか、なぜ起きたのかを後から確認できるようにするのが目的です。記録したセッションは .webblackbox というアーカイブにまとめられ、タイムライン、スクリーンショット、ネットワーク、コンソール、ストレージ、DOM、パフォーマンスの文脈を保ったまま再生できます。バグ報告の際に「再現手順が分からない」「環境依存で再現しない」といった問題を、記録そのものを渡すことで解消しようとするツールです。

主な機能

記録対象は、ユーザー入力とナビゲーション、コンソールログと実行時エラー、リクエスト・レスポンス・失敗・リダイレクト・WebSocket・SSE などのネットワークイベント、DOM の変更とスナップショット、スクリーンショットとビューポート変化、Cookie・localStorage・sessionStorage・IndexedDB・Cache・Service Worker のライフサイクル、Web Vitals・long task・トレース・CPU プロファイル・ヒープスナップショットなどです。記録モードはページスレッドへの負荷を抑えた lite と、Chrome DevTools Protocol（CDP）を使ってより詳細に取る full から選べます。Player では curl、fetch、HAR、Playwright スクリプ

ト、バグレポートといった成果物を生成でき、ファイル一覧からは GitHub Issue や Jira 向けテンプレート、セッション比較、DOM 差分などの機能もあることがうかがえます。

仕組み・アーキテクチャ

pnpm と turbo によるモノレポ構成です。apps 配下に Chrome 拡張（extension）、再生 UI（player）、MCP サーバー（mcp-server）、共有リンク用サーバー（share-server）があります。packages 配下にはイベントスキーマやメッセージ型を定義する protocol、CDP 通信を扱う cdp-router、リングバッファや秘匿化（redaction）を持つ recorder、チャンク化・インデックス作成・エクスポート・プライバシー処理を担う pipeline、アーカイブ解析用の player-sdk、アプリ埋め込み用の webblackbox があります。アーカイブは ZIP 形式で、manifest.json、チャンク化された NDJSON のイベントストリーム、時刻・リクエスト・テキストのインデックス、スクリーンショットや DOM スナップショット等のコンテンツアドレス型 blob、検証用の整合性ハッシュで構成されます。暗号化には AES-GCM と PBKDF2 による鍵導出を使い、manifest は読める状態に保たれます。

技術構成

主要言語は TypeScript で、Node.js 22 以上が必要です。ビルドには tsup、テストには vitest と @vitest/coverage-v8、Lint と整形には ESLint（typescript-eslint）と Prettier を使っています。バージョン管理とリリースは Changesets、コミット規約は commitizen と cz-conventional-changelog、pre-commit フックは husky と lint-staged です。Player SDK の API ドキュメントは typedoc で生成されています。Player には button.tsx などの UI コンポーネントや shell.tsx があり TSX を使っていますが、具体的な UI フレームワーク名は README からは不明です。ベンチマークの CI 閾値やバンドルサイズ予算の設定ファイルも含まれ、性能面の回帰検知を意識した構成になっています。

使い方・導入方法

既存アーカイブを見るだけなら、ホスト版 Player（https://webllm.github.io/webblackbox/）を開いて Load Archive から .webblackbox ファイルか互換 ZIP を選びます。記録には Chrome 拡張を使い、Chrome Web Store か GitHub Releases から入手します。README では、パッケージを展開して chrome://extensions/ で Developer mode を有効にし、Load unpacked で読み込む手順が書かれています。ツールバーアイコンから Start Lite または Start Full を選び、問題を再現してからアーカイブを書き出します。自分のアプリに組み込む場合は npm install webblackbox の後、WebBlackboxLiteSdk を new して start() を呼ぶだけで lite 記録を始められます。

注目ポイント・注意点

記録から再生、解析ツール作成、AI 連携、共有サーバーのセルフホストまでを一通りそろえている点が目立ちます。ドキュメントにはプライバシーモデル、セキュリティ概要、エンタープライズ管理者ガイドが用意されており、pipeline や recorder にも秘匿化処理があります。一方で、ネットワークのボディや Cookie、ストレージまで記録できるため、アーカイブに機密情報が含まれる可能性があり、共有時は暗号化や秘匿化設定の確認が欠かせません。full モードは CDP を使うため、ページへの影響や詳細度は lite と異なります。作成は2026年2月と新しく、スター数もまだ少ないため、API の安定性や長期的な保守状況は README からは不明です。

技術構成: TypeScript、Node.js 22+、pnpm、turbo、tsup、vitest、ESLint、typescript-eslint、Prettier、Changesets、husky、lint-staged、commitizen、typedoc、Chrome Extension、Chrome DevTools Protocol (CDP)、MCP (Model Context Protocol)、AES-GCM / PBKDF2
向いている人: 再現しにくいバグの調査に困っているフロントエンドエンジニアや QA 担当者、ユーザー環境で起きた不具合を開発チームに正確に引き継ぎたいサポート担当者に向いています。記録から Playwright テストやバグレポートを作りたいチーム、AI アシスタントにセッション解析を任せたい人、記録・再生の仕組みや CDP 活用、Chrome 拡張のモノレポ設計を学びたい学生やエンジニアにも参考になります。

テーマ 15. 116-Labs/OfflineCV

https://github.com/116-Labs/OfflineCV

★ 11 ／ フォーク 3 ／ 言語 TypeScript ／ 作成 2026-05-29 ／ Apache-2.0
該当テーマ: WebLLM

履歴書PDFが汎用テキスト抽出器にどう読まれるかを、ブラウザ内だけで診断できるログイン不要の求職ワークベンチ

概要

OfflineCV は、履歴書PDFをドロップすると、一般的なテキスト抽出器が実際に読み取った内容と、構造・具体性に関する独自スコアを表示するツールです。2カラムレイアウトの読み順の崩れ、画像だけのPDF、フォントのエンコードが文字に戻らないPDFなど、気づきにくい失敗パターンを見える化します。処理はすべてブラウザ内で行われ、PDFはアップロードされません。さらに、求人票（JD）とのマッチング、求人検索、応募管理、WebGPU による端末内AIでの書き換えといった機能も備えています。

要点

- PDFを外部に送らず、ブラウザ内だけで解析する
- 2カラム・画像のみ・フォント非対応PDFの失敗を検出する
- スコアは独自の参考値で、特定ATSの評価ではない
- WebLLMによる端末内AIでJD照合や書き換えができる
- 静的サイトとしてビルドでき、Apache-2.0で公開

何をするものか

履歴書PDF向けの「パーサーのストレステスト」です。README では、診断のためのツールであって、こうすべきと指示するものではないと説明されています。ツール自身のパーサーが何を読み取ったかを示すことで、後段のシステムが目にする前に候補者のテキストが失われてしまうケースを見つけられます。主な対象は、抽出器が横方向に読んでしまう2カラムレイアウト、選択できるテキストがゼロの画像PDF、Framer・Affinity・一部の InDesign 書き出しに見られる、フォントのエンコードが文字に復元できないPDFです。最後のケースでは、PDFに埋め込まれたリンク注釈を手がかりとして回収し、何が残ったかを確認できるようにしています。

主な機能

解析結果をもとに履歴書を再構成して表示し、インライン編集やエクスポートができます。JD を貼り付けると、キーワードベースの照合か、WebLLM モデルを使ったセマンティック照合（要件を抽出し、満たしている・一部満たしている・不足しているを判定）が行われます。Find jobs タブでは、Remotive・Arbeitnow・Jobicy の公開フィードから求人を取得し、履歴書と照らしてランキングします。このリクエストには導出した検索キーワードだけが含まれ、履歴書本文は送られません。ほかに、IndexedDB を使った履歴書ライブラリ、求人トラッカー、カバーレター関連の画面、フィードバックダイアログがあります。

仕組み・アーキテクチャ

pdfjs-dist でテキストを抽出し、runCascade と呼ばれるパイプライン、スコアリング、箇条書きのグルーピングを経て UI に描画します。ビルドは Vite のマルチエントリー構成で、/（パーサー監査）と /jobs（求職ワークベンチ）の2ページを静的な dist/ に出力します。データ保存には localStorage の UX フラグと、IndexedDB の offlinecv データベース（resumes と jobs の2ストア）を使います。履歴書が自動保存されるのは初めて編集したあとだけで、JSON によるエクスポート・インポートでバックアップできます。packages/core というワークスペースパッケージもありますが、その役割の詳細は README からは不明です。

技術構成

TypeScript、React 19、Vite 7、Tailwind CSS 4 で構成されています。PDF処理には pdfjs-dist、pdf-lib、@pdf-lib/fontkit、端末内LLMには @mlc-ai/web-llm、Word 文書の変換には mammoth と turndown、電話番号の処理には libphonenumber-js、ストレージには idb を使用しています。テストは vitest（jsdom）と Playwright による Chromium レイアウトテストで、静的解析に fallow を使います。デザインシステムは CSS カスタムプロパティによるトークン層と、@design-system エイリアスによるコンポーネント層に分かれており、フォークしなくてもブランドやコンポーネントを差し替えられる設計です。

使い方・導入方法

npm install のあと npm run dev を実行すると、自己署名証明書つきの HTTPS で localhost:5173 が起動します。WebGPU はセキュアコンテキストでしか使えないため、HTTPS が既定になっています。LAN 上の別マシンでデモする場合は npm run dev:http を使います。この場合 AI 機能は WebGPU 非対応として扱われますが、それ以外の機能は動作します。npm run fidelity にPDFやディレクトリを渡すと、UI が描画する内容や重複などをターミナルに出力でき、パースのバグ調査に使えます。npm run verify は CI と同じチェックを実行し、pre-push フックとしても自動でインストールされます。npm run build で生成した dist/ は、任意の静的ホスティングに配置できます。

注目ポイント・注意点

ステータスはアルファ版です。スコアはパーサー改善のための独自の参考値であり、特定の ATS がどう評価するかを主張するものではありません。テレメトリは既定で無効で、VITE_POSTHOG_KEY を設定してビルドしたとき（offlinecv.org など）だけ PostHog にイベントが送られます。送信されるのはファイルサイズや件数などで、PDF の中身やテキストは含まれないと明記されています。ただし、GitHub のスター数取得や求人フィードへのアクセスでは、利用者の IP アドレスが第三者に伝わる点に注意が必要です。Linux で WebGPU を使うには、Vulkan を有効にする必要がある場合があります。.claude/skills や各種 GitHub Actions など、AI エージェントを活用した開発運用の仕組みが充実している点も特徴です。

技術構成: TypeScript、React 19、Vite 7、Tailwind CSS 4、pdfjs-dist、pdf-lib、@mlc-ai/web-llm、WebGPU、idb (IndexedDB)、mammoth、turndown、libphonenumber-js、posthog-js（任意）、Vitest、Playwright、ESLint、fallow
向いている人: 自分の履歴書PDFが採用システムや抽出ツールに正しく読まれるかを、プライバシーを保ったまま確認したい求職者に向いています。PDF解析、クライアントサイドだけで動くアプリ、ブラウザ内LLM（WebLLM/

WebGPU）の実装例を学びたいフロントエンドエンジニアや学生にも参考になります。Apache-2.0 なので、履歴書解析機能を自社プロダクトに組み込みたい開発者にも適しています。

テーマ 16. TechMitten/Origami-AI

https://github.com/TechMitten/Origami-AI

★ 11 ／ フォーク 2 ／ 言語 TypeScript ／ 作成 2025-12-27 ／ MIT
該当テーマ: WebLLM

PDFスライドをAIナレーション付き動画に変換するなど、WebGPUでAI処理をすべてブラウザ内で完結させるAIスタジオ

概要

Origami AIは、PDFのスライド資料からAIでナレーション原稿を作り、音声合成して動画化する処理を、ブラウザ内だけで行えるWebアプリです。WebGPU上で動くWebLLMとKokoro.js、FFmpeg.wasmを組み合わせているため、データのアップロードやレンダリングサーバー、サブスクリプションを必要としません。当初はPDF動画化ツールでしたが、現在はショート動画生成、自動ズーム付き画面録画、ローカルAIチャット、動画のシーン分析まで扱うプラットフォームに広がっています。教材やプレゼン動画を手軽に作りたい人や、プライバシーを重視したいクリエイターに向いています。

要点

- PDFから台本・音声・MP4動画まですべてブラウザ内で生成
- WebLLMでローカルLLM推論、Kokoro.jsで音声合成を実行
- FFmpeg.wasmで720p/1080pのMP4をクライアント側でレンダリング
- ショート動画生成、画面録画、AIチャット、動画シーン分析も搭載
- APIキー不要で動き、OpenAI互換APIやGeminiも任意で利用可能

何をするものか

Origami AIは、ブラウザで動く「AIスタジオ」です。中心となる機能は、PDFのプレゼン資料をアップロードすると、スライドごとにAIがナレーション原稿を書き、音声合成で読み上げを作り、BGMやトランジションを付けてMP4動画に書き出すというものです。これらの処理は基本的に利用者の端末上で行われ、クラウドへのデータ送信やAPI呼び出しは必須ではありません。ほかにも、トピックから縦型ショート動画を作る機能、操作に合わせて自動でズームする画面録画、画像や動画を添付できるローカルAIチャット、MP4をタイムスタンプ付きのシーンに分ける分析機能を備えています。ホスト版がhttps://origami.techmitten.com/で公開されているため、インストールせずに試すこともできます。

主な機能

PDF動画化では、pdfjs-distでスライド画像とテキストを取り出し、LLMで原稿を生成し、Kokoro.jsで複数の声から選んで音声にします。BGMはナレーション中に自動で音量を下げるダッキングに対応し、スライドごとに調整できます。ショート動画生成（/shorts）では、15〜90秒の長さ、トーン、9:16・16:9・1:1のアスペクト比、ビジュアルスタイルを指定します。シーンごとの台本と画像プロンプトを確認したうえで、Pollinations経由の画像・動画生成とTTSを並行して実行し、Bold Pop、Karaoke Fill、Clean Lower Thirdのいずれかのスタイルで字幕を焼き込みます。作成したプロジェクトは、スライド・メディア・音声・設定をまとめた.origamiアーカイブとしてエクスポートとインポートができます。

仕組み・アーキテクチャ

フロントエンドはReactで構成され、src/servicesにWebLLM、TTS、FFmpegレンダラー、ストレージ、アーカイブ処理などがまとまっています。webLlm.worker.tsやtts.worker.tsというファイルがあることから、重い推論処理はWeb Workerに分けていると見られます。レンダリングはBrowserVideoRenderer.tsやShortsVideoRenderer.tsが担当し、ffmpegFilterGraph.tsやwebCodecsEncoding.tsも含まれています。バックエンドは、ローカル用のExpress 5サーバー（server.ts、Viteミドルウェア付き）と、Cloudflare Pages Functions（functions/api/）の2系統です。後者にはLLMチャット、動画分析、Pollinations画像・動画のプロキシなどがあります。FFmpeg.wasmを動かすにはCOOP/COEPヘッダーが必要なため、npm run devなど決められたコマンドで起動する必要があります。APIキーはブラウザのIndexedDBに保存され、サーバーには保存されないと説明されています。

技術構成

言語はTypeScriptで、React 19、Vite 7、Tailwind CSS 4、React Router 8を使っています。AI推論は@mlc-ai/web-llm、音声合成はkokoro-js、動画合成は@ffmpeg/ffmpeg、PDF処理はpdfjs-distが担います。UIではドラッグ&ドロップに@dnd-kit、アイコンにlucide-reactを使っています。package.jsonには、ほかに@huggingface/transformers、tesseract.js（OCR）、jszip、jspdf、html2canvas、firebase、react-markdown、@marsidev/react-turnstileも含まれます。サーバー側はexpress、helmet、hpp、express-rate-limit、corsなどでセキュリティを固めています。ローカルLLMとしては、Gemma 2 2B、Llama 3.2 1B/3B、画像・動画解析に対応したマルチモーダルのPhi 3.5 Visionが挙げられています。

使い方・導入方法

最も手軽なのはホスト版を使う方法です。ローカルで動かす場合は、Node.jsとWebGPU対応ブラウザを用意し、git cloneしたあとnpm installとnpm run devを実行して、http://localhost:3006を開きます。Node.jsの要件はREADMEでは20.19.0以上とされていますが、package.jsonのenginesは22.16.0以上になっているため、新しいバージョンを使うほうが無難です。Dockerを使う場合はdocker compose up --buildで起動し、ポート3000で利用できます。画面録画で正確な自動ズームを使いたいときは、chrome-extension/フォルダをChromeの「パッケージ化されていない拡張機能」として読み込みます。設定画面では、Kokoroの量子化（q8とq4）、WebLLMで使うモデルと精度、OpenAI互換API（Gemini、OpenRouter、Groq、Ollamaなど）、ナレーション用プロンプトを変更できます。

注目ポイント・注意点

推論から動画エンコードまでをブラウザだけで行う構成は、プライバシーとコストの面で大きな利点があります。初回にモデルをダウンロードすれば、その後はオフラインでも動くとされています。一方で、ハードウェアの要件はそれなりに高く、最低でも8GB RAMと内蔵GPU、推奨は16GB RAMとF16対応の専用GPUです。モデルのダウンロードサイズも約0.8〜3.9GBあります。対応ブラウザはChrome/Edge 113以降が中心で、FirefoxはNightlyで設定を有効にする必要があります。ショート動画の画像・動画生成は外部サービスのPollinationsに依存しています。また、Firebaseの設定やAuthModalがあり、認証やクラウド同期の機能も含まれているようですが、その範囲はREADMEからは不明です。スター数は少なく、まだ若いプロジェクトです。

技術構成: TypeScript、React 19、Vite 7、Tailwind CSS 4、React Router 8、@mlc-ai/web-llm、kokoro-js、@ffmpeg/ffmpeg (FFmpeg.wasm)、pdfjs-dist、@huggingface/transformers、tesseract.js、@dnd-kit、lucide-react、Express 5、Cloudflare Pages Functions (wrangler)、Firebase、WebGPU、Docker
向いている人: PDFの講義資料やプレゼンからナレーション付き動画を作りたい教育関係者やエンジニア、ショート動画を手早く作りたいコンテンツ制作者に向いています。データを外部に送りたくないプライバシー重視の人にも合います。また、WebGPU、WebLLM、FFmpeg.wasmを組み合わせたブラウザ完結型AIアプリの実装例を学びたいフロントエンド開発者や学生にとっても参考になります。

テーマ 17. webllm-org/ai-blocks

https://github.com/webllm-org/ai-blocks

★ 10 ／ フォーク 2 ／ 言語 TypeScript ／ 作成 2025-12-04
該当テーマ: WebLLM

WebLLM上で動く、サーバーもAPIキーも不要なReact向けAIコンポーネント集（shadcn形式）

概要

ai-blocksは、ブラウザ内で言語モデルを扱うプロトコル「WebLLM」を前提にした、AI機能付きUIコンポーネントのコレクションです。チャット、テキスト生成、構造化出力、要約・翻訳、フォーム、EC、教育、アクセシビリティなど50以上のコンポーネントを、shadcn CLIで1つずつ自分のプロジェクトに追加できます。サーバー側のAPI呼び出しを使わず、プライバシーに配慮したAI機能をフロントエンドだけで組み込める点が特徴です。React＋shadcn/ui＋Tailwind CSSの環境でAI UIを手早く試したい開発者を想定しています。

要点

- WebLLM向けのAI UIコンポーネントを50種類以上収録
- shadcn CLIで個別追加、またはnpmパッケージで導入
- サーバー不要・APIキー不要をうたうブラウザ内AI
- 現状はReact 18以上＋shadcn/ui＋Tailwind CSS限定
- use-chatなどのフックや基本UI部品も同梱

何をするものか

ai-blocksは、AIを使ったフロントエンド画面を作るためのUIコンポーネント集です。土台となるWebLLMは、ブラウザ上で言語モデルにアクセスするための統一APIを提供する「ブラウザAI統合プロトコル」と説明されており、サーバー側でAPIを呼ばずにAI機能を実現することを目指しています。ai-blocksはその上に、チャット画面や文章生成、感情分析、エンティティ抽出、要約、翻訳などをすぐに使える形でまとめたものです。各コンポーネントは公式サイトのwebllm.org/blocksでライブプレビューやソースコードとともに閲覧できます。

主な機能・収録コンポーネント

READMEでは、シンプルチャット・ペルソナ・音声チャット・ツール利用付きチャットなどのチャット系、ストリーミング・書き換え・展開・オートコンプリートなどの文章生成系、感情分析・エンティティ抽出・JSON生成といった構造化出力系が紹介されています。さらに要約や翻訳、読みやすさ調整、スマートバリデーションやオートフィル付きフォーム、商品検索やサイズアドバイザーなどのEC向け、フラッシュカードやクイズ、ソクラテス式チューターなどの教育向け、代替テキスト生成や文章の簡略化などアクセシビリティ向けのものもあります。ファイル一覧にはCommitMessageWriterDemo、RegexGeneratorDemo、ImageGenerationDemoなど遊び心のあるものも含め、多数のデモが並んでいます。

仕組み・アーキテクチャ

ソースはsrc/registry以下に整理されており、blocks（chat-interface、prompt-form、streaming-text、availability、demosなど）、hooks（use-chat、use-streaming、use-webllm）、ui（ai-avatar、ai-input、chat-bubble、message-list、cloud-login-buttonなど）の3層構成になっています。availabilityディレクトリにはAvailabilityBadgeやCapabilityGate、CloudAuthBanner、OAuthDomainAuthといったデモがあり、AI機能の利用可否表示や認証まわりも扱っているようですが、詳細な挙動はREADMEからは不明です。scripts配下のgenerate-registry-from-blocks.jsやbuild-registry.jsでshadcn用のregistry.jsonを生成し、extract-props.cjsでreact-docgen-typescriptを使ってprops情報を抽出する仕組みになっています。

技術構成

主要言語はTypeScriptで、ビルドにはtsupを使用しています。ランタイム依存はclsxとtailwind-mergeのみと軽量で、react、react-dom、lucide-react、webllmはpeerDependenciesとして扱われます（webllmとlucide-reactはoptional指定）。スタイリングはTailwind CSS、UIの土台はshadcn/uiで、アイコンにはlucide-reactを使います。package.jsonのexportsではルートのほか、demos/*を個別にimportできるパスや、registry/*、lib/*のソースを直接参照できるパスが定義されています。package.jsonのライセンス欄はMITですが、リポジトリのメタデータ上は不明となっています。

使い方・導入方法

推奨はshadcn CLIで、npx shadcn add @ai-blocks/simple-chat のように必要なコンポーネントだけを追加します。npx shadcn@latest add https://webllm.org/r/simple-chat.json のようにURL指定でも追加可能です。もう一つの方法はnpmで、npm install ai-blocks webllm を実行し、import { SimpleChatDemo } from 'ai-blocks' または 'ai-blocks/demos/SimpleChatDemo' から読み込みます。ただし直接importする場合も、shadcn/uiのセットアップと @/components/ui/* や @/lib/utils のパスエイリアス設定が必要です。前提環境はReact 18以上、webllmパッケージ、Tailwind CSS、lucide-reactです。

注目ポイント・注意点

shadcn方式でソースコードごと自分のプロジェクトに取り込めるため、見た目や挙動を自由にカスタマイズしやすい点が魅力です。一方で、現時点ではReact＋shadcn/uiのみ対応で、素のReactやvanilla JS向けのエクスポートは将来追加される可能性がある段階とされています。バージョンは0.0.1と初期段階で、スター数も少なく、package.jsonのrepositoryはwebllm-org/webllmモノレポのpackages/ai-blocksを指しています。実際にどのモデルがどのように動作するか、対応ブラウザや性能要件などはWebLLM側の仕様に依存し、READMEからは不明です。

技術構成: TypeScript、React、shadcn/ui、Tailwind CSS、WebLLM (webllm)、lucide-react、clsx、tailwind-merge、tsup、react-docgen-typescript
向いている人: React＋shadcn/ui＋Tailwind CSSでWebアプリを作っており、サーバーやAPIキーを用意せずにチャットや要約、翻訳、フォーム補助などのAI機能を手早く組み込みたいフロントエンド開発者。ブラウザ内AIのUIパターンを多数のサンプルから学びたい学生やプロトタイプ制作者にも向いています。

テーマ 18. kyrillosishak/Domicile

https://github.com/kyrillosishak/Domicile

★ 10 ／ フォーク 2 ／ 言語 TypeScript ／ 作成 2025-11-17 ／ MIT
該当テーマ: WebLLM

ベクトルDB・埋め込み・RAG・ローカルLLMをすべてブラウザ内で動かし、データを端末外に出さないプライベートAIスタック

概要

Domicile は、文書の保存、埋め込み生成、ベクトル検索、RAG による回答生成までをブラウザ内で完結させる TypeScript ライブラリです。クラウドや外部 API を使わないため、データが端末の外へ送信されない（zero egress）構成を、設定ではなくアーキテクチャそのもので保証することを狙っています。主な想定ユーザーは機密文書を扱う法律事務所などのチームと、顧客にオンプレ相当のデータ保護を提供したいインテグレーターです。WebGPU と WASM の両対応、引用付き RAG、MCP サーバー機能、Tauri 製デスクトップアプリまでを一つのスタックにまとめている点が特徴です。

要点

- npm パッケージ @kyrillosishak/domicile として配布されるブラウザ完結型 AI スタック
- IndexedDB と純 TypeScript 実装の HNSW で 10 万件超の文書検索をうたう
- WebLLM（GPU）と Wllama（CPU）を自動で切り替えるデュアル LLM 構成
- BM25 とのハイブリッド検索、リランカー、引用付き RAG に対応
- MCP サーバー、React hooks、Tauri デスクトップアプリも同梱

何をするものか

Domicile は、ベクトルデータベース、ローカル埋め込み、ローカル LLM、RAG パイプラインをまとめてブラウザ上で動かすプライベート AI スタックです。初回にモデルの重みを読み込んだ後はオフラインでも動作し、文書やクエリが第三者のサーバーへ送られることはないと README は説明しています。API キーや席単位の課金、クラウド費用もかかりません。想定用途として、弁護士と依頼者間の秘匿特権に関わる文書を扱う法務チームが挙げられています。また、オンプレ設備への投資をせずにオンプレ相当のデータ保管を顧客に提供したいインテグレーション・インフラチームも対象です。README では GDPR 準拠を「構造上満たす」と表現していますが、これは作者の主張であり、法的な適否は利用者側で判断する必要があります。

主な機能

ベクトル検索は IndexedDB への永続化と純 TypeScript の HNSW グラフインデックスで実装されています。cosine、euclidean、dot の各距離に対応し、インデックスを再構築せずに削除できます。メタデータフィルタ（例えば field、operator、value による eq 指定）も使えます。埋め込み生成には Transformers.js を使い、WebGPU が使えない環境では WASM にフォールバックします。モデルの重みは端末内にキャッシュされます。RAGPipelineManager は検索結果をもとに回答を生成し、回答の根拠となった文書を result.sources として返します。そのほか、ロードマップ上で実装済みとされている機能は次のとおりです。BM25 と dense 検索を reciprocal-rank fusion で統合するハイブリッド検索、cross-encoder によるリランク、CLIP によるテキス

ト・画像のマルチモーダル埋め込み、fp16・int8・int4 の量子化、useSearch や useRag などの React hooks です。

仕組み・アーキテクチャ

README はスタックを 6 つの層に分けて説明しています。Interface 層は、アプリ向けの TypeScript API と AI エージェント向けの MCP です。Intelligence 層は RAG Pipeline Manager と LLM ランタイムで、WebLLM と Wllama を GPU から CPU へ自動フォールバックさせます。Retrieval 層は Transformers.js、HNSW、BM25、Reranker から成ります。Custody 層は IndexedDB を管理し、容量を考慮した退避や、JSON・バイナリ形式でのエクスポートとインポートを担います。Acceleration 層は WebGPU、WASM SIMD、ワーカープールで構成されます。Reliability 層には型付きエラー階層、LRU キャッシュ、メモリマネージャ、ベンチマークランナーが含まれます。ソースツリーには residency.ts というファイルがあり、デスクトップ版の説明では、モデル重みの初回ダウンロードを除いてネットワーク送信がゼロであることを確認する Residency guard に触れています。

技術構成

言語は TypeScript で、ESM 形式のパッケージとして配布されます。主な依存ライブラリは次のとおりです。埋め込みには @huggingface/transformers と onnxruntime-web、GPU 推論には @mlc-ai/web-llm、MCP 対応には @modelcontextprotocol/sdk を使います。依存には hnswlib-wasm と fake-indexeddb も含まれています。CPU 推論用の @wllama/wllama は任意の peerDependency です。ビルドには Vite と vite-plugin-wasm、テストには Vitest と happy-dom、E2E 系には Playwright、API ドキュメント生成には TypeDoc を使っています。デスクトップ版は Tauri（Rust）製で、Chromium を同梱しない約 10MB のラッパーとされています。CLI も同梱されており、domicile コマンドで bench、capabilities、export、import、init、serve を実行できます。また、PyScript 向けの別パッケージ domicile-pyscript や、legal・health 向けのプロファイル JSON を含む integrator-pack もあります。

使い方・導入方法

npm install @kyrillosishak/domicile でインストールします。次に、storage（dbName）、index（dimensions、metric）、embedding（model、device）を指定して new Domicile を作成し、initialize を呼びます。あとは insert で文書を追加し、search で検索するのが基本の流れです。RAG を使う場合は、WllamaProvider などの LLM プロバイダと RAGPipelineManager を組み合わせ、rag.query に topK や生成オプションを渡します。MCP サーバーとして使う場合は、new MCPServer(db, rag) を作成し、mcp.serve({ transport: 'stdio' }) を呼ぶと Claude Desktop などから利用できるとされています。開発用には npm test、npm run build、npm run benchmark などのスクリプトが用意されています。デスクトップ版は、ライブラリをビルドしたあと desktop ディレクトリで npm run tauri:build を実行します。

注目ポイント・注意点

ベンチマーク結果も公開されています。10K ベクトル（128 次元、cosine）の検索で p50 2.4ms、recall@10 = 0.91、法務文書の既知解答コーパスでの引用精度は recall@3 = 0.92 です。ただし、これは Linux サーバー CPU 上の Node で測定した値で、ブラウザで WebGPU を使う場合は結果が異なると README 自身が注記しています。ブラウザの対応状況にも注意が必要です。WebGPU は Chrome・Edge の 113 以降のみで、Firefox と Safari では未対応とされています。マルチスレッド化には SharedArrayBuffer が必要です。スター数 10 の若

いプロジェクトで、バージョンは 0.2.2 のため、API が今後変わる可能性があります。ローカル LLM の推論速度やメモリ消費の実測値は README からは不明です。

技術構成: TypeScript、IndexedDB、WebGPU、WebAssembly (WASM SIMD)、@huggingface/transformers (Transformers.js)、onnxruntime-web、@mlc-ai/web-llm、@wllama/wllama、hnswlib-wasm、@modelcontextprotocol/sdk、React、Vite、Vitest、Playwright、TypeDoc、Tauri (Rust)
向いている人: 機密文書をクラウドに出せない法律事務所・法務チームや医療など規制の厳しい業界の開発者、顧客アプリに組み込める端末内完結の RAG・検索基盤を探しているインテグレーター、そして WebGPU や WASM でブラウザ内 LLM・ベクトル検索を試したいフロントエンドエンジニアや学生に向いています。

テーマ 19. soulbah/regeste

https://github.com/soulbah/regeste

★ 9 ／ フォーク 1 ／ 言語 TypeScript ／ 作成 2026-08-06 ／ AGPL-3.0
該当テーマ: WebLLM

文書の解析から検索・回答生成までをブラウザ内で完結させる、プライバシー重視の文書チャット（RAG）アプリ

概要

Regeste は、機密性の高い文書を手元から出さずに AI と対話するためのオープンソースの文書チャットアプリです。文書の解析、チャンク分割、埋め込み生成、ベクトル検索、チャット履歴の保存はすべてブラウザ内で動きます。回答ごとに「Private」「Assisted」「My AI」の3モードから、AI にどこまで情報を見せるかを選べます。サーバー側は認証・クォータ・Assisted 用エンドポイントを担う Cloudflare Worker 1つだけで、Private と My AI は静的クライアントとして動作します。

要点

- 解析・埋め込み・ベクトル検索・履歴をすべてブラウザ内で処理
- Private / Assisted / My AI の3モードで送信範囲を選べる
- Assisted では AI に送った抜粋を「What AI saw」で確認可能
- SvelteKit と SQLite WASM + sqlite-vec によるローカル RAG
- AGPL-3.0 で公開され、セルフホストも可能

何をするものか

Regeste は、契約書のような機密文書をアップロードして質問し、引用付きの回答を得るための文書チャットアプリです。README は「ファイルはローカルに残り、AI に何を見せるかは利用者が選ぶ」と説明しています。デモでは、賃貸契約の PDF を2つ添付して質問すると、該当箇所を引用した回答が返り、その間データは端末の外に出ません。ホスト版は regeste.com で試せます。

主な機能：3つの信頼境界

回答ごとに、どこまでデータを出すかを3段階から選びます。Private は WebGPU/WASM で LLM を端末上で動かすモードで、文書も質問も外部に送られません。モデルの重みは公開ファイルで、同一オリジン経由で一度だけダウンロードされ、ブラウザにキャッシュされます。Assisted は検索で取り出した抜粋だけを Cloudflare Worker（Workers AI）へ一時的に送るモードで、送った内容は保存されません。実際に送られた内容は「What AI saw」で確認できます。My AI は Ollama、LM Studio、vLLM など OpenAI 互換の自前エンドポイントを使うモードです。Assisted は既定で無効で、PUBLIC_ASSISTED_ENABLED=true を設定したときだけ表示されます。

仕組み・アーキテクチャ

アプリ本体は静的クライアントで、ブラウザ内の SQLite WASM と sqlite-vec でベクトル検索を行います。依存関係を見ると、PDF 処理に pdfjs-dist と @llamaindex/liteparse-wasm、Word ファイルに mammoth、OCR に ppu-paddle-ocr と ppu-doclayout、埋め込みに @huggingface/transformers、オンデバイス LLM に @mlc-ai/web-llm と @wllama/wllama が使われています。Web Worker との通信には comlink が使われています。サーバー側は Cloudflare Workers、D1、Workers AI で構成され、better-auth と drizzle で認証とクォータを管理します。詳しい構成は docs/ARCHITECTURE.md にまとめられています。

技術構成と開発体制

フロントエンドは SvelteKit（Svelte 5）、Tailwind CSS v4、shadcn-svelte で、バリデーションには valibot を使っています。ランタイムは Bun 1.2 と Node 22 です。テストは vitest と Playwright で行います。src/lib/benchmark や benchmarks/ には検索精度やファジー検索のベンチマークがそろっていて、retrieval の品質を数値で追っていることがうかがえます。specs/ 配下には機能ごとに spec・plan・tasks を書く仕様駆動の開発記録があります。また、AGENTS.md、.agents/skills/、.claude/ に AI コーディングエージェント向けの仕組みが用意されており、AI エージェントと協働して開発されていることが明記されています。

使い方・導入方法

開発環境では、bun install で依存関係を入れ、.dev.vars.example を .dev.vars にコピーします。続いて bun run db:migrate:local でローカル D1 をマイグレーションし、bun run dev で起動します。dev では Cloudflare バインディングがエミュレートされ、Assisted モードもローカルで有効になります。bun run verify を実行すると、型チェック、lint、テスト、ビルドをまとめて行えます。Private と My AI だけをセルフホストする場合は、認証やデータベースの設定は不要です。デプロイは wrangler deploy で行います。

注目ポイント・注意点

「Don't trust us」という方針のもと、コードを検証できること、セルフホストできること、完全にローカルで使えることを重視しています。ライセンスは AGPL-3.0-only で、改変版をサービスとして提供する場合は変更点の公開が必要です。現在は 0.x のプレリリース段階で、README は「何がいつ変わってもおかしくない」と明記しています。リリースは semantic-release で自動化されています。Issue には文書の内容や個人情報を貼らないよう注意書きがあります。対応ブラウザやモデルの具体的な要件は README からは不明です。

技術構成: TypeScript、SvelteKit (Svelte 5)、Tailwind CSS v4、shadcn-svelte、Cloudflare Workers / D1 / Workers AI、better-auth、drizzle-orm、valibot、SQLite WASM + sqlite-vec、@mlc-ai/web-llm、@wllama/wllama、@huggingface/transformers、pdfjs-dist、mammoth、ppu-paddle-ocr、Bun、Vite、Vitest、Playwright
向いている人: 機密文書を外部クラウドに送らずに AI で扱いたい個人や組織に向いています。また、ブラウザ内で完結する RAG（WebGPU/WASM による LLM、ブラウザ内ベクトル検索）の実装を学びたいエンジニアや学生、Ollama などの自前 LLM と組み合わせて文書チャットをセルフホストしたい人、仕様駆動と AI エージェントを組み合わせた開発手法に関心のある開発者にも参考になります。

テーマ 20. KREASIOKA/KREASYS

https://github.com/KREASIOKA/KREASYS

★7 ／ フォーク1 ／ 言語 JavaScript ／ 作成 2026-02-28 ／ MIT

該当テーマ: WebLLM

ブラウザだけで動くサーバーレスなAIエージェント用IDE。WebGPUによるローカルLLM推論とTelegram連携に対応する

概要

KREASYSは、バックエンドもビルド工程も持たず、ブラウザ内だけで完結するAIエージェント向けIDEです。ファイル保存やメモリ管理、AIモデルの振り分けといった本来サーバーが担う処理を、IndexedDB、WebGPU、Service Workerなどのブラウザ機能で代わりに実現しています。AIは応答の中にXMLタグを出力し、パーサーがそれを読み取って、仮想ファイル書き込み、HTMLの画像化、サブエージェントの起動、Telegram送信、iframeブラウザ操作などを実行します。プライバシーを重視し、ローカルで動くエージェント環境を手軽に試したい開発者に向いたAlpha版のプロジェクトです。

要点

- 100%クライアントサイドで動き、ビルド不要、静的サーバーで起動できる
- @mlc-ai/web-llmを使い、WebGPUでローカルLLM推論ができる
- AIが出力するXMLタグでファイル操作やブラウザ操作を実行する
- OpenRouter、Anthropic、Ollamaなど複数のプロバイダに対応
- Telegram Bot連携と2層の永続メモリを備えたAlpha版

何をするものか

KREASYSは、ブラウザ上で完結する自律型AIエージェント用の統合環境(IDE)です。README によると、バックエンド、ビルドステップ、管理対象のサーバーはいずれもありません。仮想ファイルシステム(VFS)、永続メモリ、複数プロバイダ間のモデルルーティング、Telegram連携、ローカルLLM推論、AIが操作できるブラウザパネルを1つのシングルページアプリにまとめています。中心にある考え方は、サーバーが担ってきたファイル保存、処理実行、メモリ、AIルーティングを、IndexedDB、WebGPU、WebAssembly、Service Workerといったモダンブラウザの機能で組み立て直せる、というものです。PWAとしてオフラインキャッシュにも対応しています。

主な機能

UIはChat、Workspace、Core Data、Models、Tracking、Browser、SysLog、Channelingの8タブで構成されています。Chatではファイルを添付して会話でき、WorkspaceではSystemとWorkspaceの2系統のVFSを編集できます。Core DataではAIの人格(personality.md)やスキル定義(skills.md)を編集し、メモリの中身を確認できます。Modelsではクラウドモデルの登録と、WebLLMによるローカルモデルのダウンロード・ロードを行います。Trackingは実行計画やログをタイムラインで表示し、Browserはサンドボックス化されたiframeとJSコンソールを提供します。ChannelingではTelegramのトークンや既知ユーザーの一覧を管理します。

仕組み・アーキテクチャ

処理はイベント駆動で進みます。ユーザーまたはTelegramからの入力を受けると、まずroute()がクエリの種類(テキスト、画像、音声、ビジョン)に応じて最適なモデルを選びます。次にllm()がSSEのストリーミングでトークンを逐次UIに表示し、最後にpsVfs()が応答内のXMLタグを解析して副作用を実行します。対応タグはfile、plan、render_html、delegate、tg_send、tg_doc、browser_open、browser_js、browser_screenshot、mediaです。メモリは2層構造で、/system/memory.logに生の活動ログ(約5万文字)を追記していきます。60秒間アイドル状態が続くと、memCompress()がこのログを要約して/system/memory.mdに長期記憶として保存し、文脈が膨らみすぎるのを防ぎます。

技術構成

素のJavaScript、HTML5、CSS3で書かれており、マニフェストファイルは存在しません。js/core配下にはstate.js(グローバル状態とシステムプロンプト)、vfs.js(VFSとタグパーサー)、ai.js(LLMルーター)、memory.js、webllm.js、telegram.jsがあります。js/ui配下にはapp.js(起動処理)とbrowser.jsが置かれています。状態の永続化にはlocalforage経由のIndexedDBを使い、キーはksaです。ローカル推論はCDNから動的importする@mlc-ai/web-llmのMLCEngineで行い、モデル一覧はprebuiltAppConfig.model_listから動的に取得します。HTMLの画像化とスクリーンショットにはhtml2canvas、アイコン描画にはlucideを使っています。

使い方・導入方法

ビルドは不要で、HTTPSかlocalhostで配信できる静的ファイルサーバーがあれば動きます。git cloneした後、リポジトリのディレクトリでpython3 -m http.server 8080を実行し、Chrome 113以降またはEdge 113以降でhttp://localhost:8080を開きます。WebGPUをフルに使うにはこれらのブラウザが必要です。クラウドモデルを使う場合は、ModelsタブでOpenRouterなどのプロバイダを追加し、APIキーとモデル名を入力します。ローカル推論を使う場合は、Modelsタブの「Local Browser Compute」でモデルを選び、Download & Loadを押します。設定はすべてIndexedDBに自動保存され、.envファイルはありません。

注目ポイント・注意点

Telegramのポーリングでは、localStorageを使った排他的なタブロックを実装しています。これにより、複数タブを開いたときに発生する409 Conflictを回避しています。一方で、現在はAlpha版であり、機能は頻繁に変わると明記されています。APIキーはブラウザのIndexedDBに保存され、browser_jsタグはAIが生成したJavaScriptをiframe内で評価します。そのため、セキュリティ面には注意が必要です。ただし具体的な対策はREADME からは不明です。また、ローカルLLMの動作に必要なGPU性能やメモリ要件も README からは不明です。テストコードの有無やライセンス以外の運用方針についての記載もありません。

技術構成: JavaScript、HTML5、CSS3、WebGPU、@mlc-ai/web-llm、IndexedDB、localforage、html2canvas、lucide、Service Worker、PWA、Telegram Bot API
向いている人: サーバーを用意せず、ブラウザだけでAIエージェントを試したい開発者に向いています。WebGPUによるローカルLLM推論やプライバシー重視のAI環境に関心がある人、Telegram Botと連携した自動化を手軽に組みたい人にも適しています。ビルド不要のバニラJS構成でエージェントの仕組み(XMLタグによるツール実行やメモリ圧縮)を学びたい学生にも参考になります。ただしAlpha版のため、本番業務での利用には注意が必要です。